

UNIVERSITY OF WATERLOO



Department of Electrical and Computer Engineering

ECE499 – Project Course

MorphIIC: An Embedded Platform for I²C Device Emulation and Hardware-in-the-Loop Validation

A Report Prepared For:

Andrew Morton & Jeff Zarnett

Simon Gorbet (20935634)

Contents

1	Report Summary	4
2	Introduction	4
3	Objective and Requirements	5
3.1	Essential Device Requirements	5
3.2	Objectives	5
3.3	Functional Requirements	6
4	Design Decisions	7
4.1	Firmware Architecture	7
4.2	Hardware Platform	8
4.3	I2C Service Emulation Strategy	9
4.4	Configuration Format	9
4.5	Dynamic data strategy	10
5	System Overview	10
6	Implementation	12
6.1	Hardware	12
6.2	I2C Service	12
6.3	Device Model	13
6.4	Dynamic Register Data Series	14
6.5	Device logs	15
6.6	CLI	15
7	Validation and Testing	16
7.1	Model Testing	16
7.2	Device Testing	18
8	Discussion	20
8.1	Testing Summary	20
8.2	Model Validation and Developer Usability	20
8.3	Dynamic Throughput and Underrun Behaviour	20
8.4	Limitations and Future Improvements	21
9	Conclusion	21
	References	21
10	Appendix	24
10.1	Appendix A: Personal Motivation	24
10.2	Appendix B: Worked Example Model Artifacts	24
10.2.1	B.1 Device model JSON	24
10.2.2	B.2 Embedded CSV source	25
10.2.3	B.3 Host-stream CSV source	25
10.2.4	B.4 Generated firmware model output	26
10.3	Appendix C: Runtime Session Outputs	28

10.3.1	C.1 Host streamer output	28
10.3.2	C.2 CLI interaction output	28
10.4	Appendix D: STM32G474 Master Test Logs	29
10.4.1	D.0 Test setup diagram	29
10.4.2	D.1 Functional profile logs	29
10.4.3	D.2 Stress profile logs	33

1 Report Summary

MorphIIC is a model-driven I2C peripheral emulator for repeatable hardware-in-the-loop validation when real devices are unavailable, delayed, or difficult to automate. In this project, I designed and implemented MorphIIC to address a need for a configurable platform that can emulate different peripherals without per-device firmware rewrites. A user-defined JSON model describes device identity, register map, access rules, and optional CSV-backed dynamic registers. Build-time tooling validates this model and generates deterministic firmware tables, allowing new device behaviours to be created through configuration instead of per-device firmware rewrites.

The implemented platform combines an RP2040 firmware runtime with a host tooling pipeline. On device, an async I2C service handles write, write-read, read, and general-call traffic against a register file with optional pointer auto-increment semantics. Dynamic data is supported in two modes: embedded CSV sequences compiled into firmware and host-streamed sequences delivered over USB into bounded per-register FIFOs. Two CDC channels are exposed concurrently for a command-line interface and streaming transport, enabling runtime observability and feeder control without reflashing.

Validation covers both model tooling and hardware execution. Host-side integration tests confirm end-to-end parse/validate/generate behaviour and expected failures for malformed models; all reported integration tests passed (3/3), and unit/property tests passed (10/10). Device-side functional tests (F01–F07) and stress tests (S01–S05) passed at both 100 kHz and 400 kHz. In high-throughput host-stream scenarios, no underrun was inferred (`equal_adjacent_pairs=0`), with representative 400 kHz timings of 575 μ s average single-register inter-read interval and 1273 μ s average 9-register burst interval.

Taken together, the results show that MorphIIC achieves the project’s core goals: configurable I2C emulation, deterministic model generation with strong build-time guardrails, and stable runtime behaviour under mixed static and dynamic workloads. Current limitations are explicit NACK fault scripting and richer inter-register behaviour modeling, which are the highest-value next steps for improving realism in advanced driver edge-case testing.

2 Introduction

MorphIIC is a configurable I2C device emulator intended to simplify hardware-in-the-loop testing of embedded systems by replacing real sensors and peripherals with reproducible, scriptable stand-ins. In this project, I designed and built MorphIIC as a practical response to the common development problem of needing realistic I2C peripherals before the final hardware is available.

Embedded systems are often validated through interaction with external peripherals such as sensors, converters, memory devices, and I/O expanders. In many cases, testing against the real device hardware can create friction. Hardware may be unavailable until later in development, difficult to distribute across teams, or inconvenient to automate at scale. Because embedded projects are cross-discipline, inconsistencies in timelines between hardware and firmware often result in major schedule risk and make bugs more difficult to pin down once discovered [1]. More generally, hardware-in-the-loop (HIL) testing is valued because it allows engineers to validate embedded software against realistic interfaces earlier in development, reducing dependence on expensive or late-arriving physical prototypes.

This problem is especially relevant for I2C-based peripherals. I2C is widely used because it only has two lines for communications and can support multiple devices on the bus which are individually addressable by their respective address. It provides an extremely economical, simple, and robust

platform for low-speed communication in embedded systems [2]. At the same time, correct operation depends on protocol details such as acknowledgements, repeated starts, and clock stretching. These behaviours are not just implementation details, they directly affect whether the firmware is able to reliably communicate with a device. Because of this, it is necessary to go beyond software mocking when testing firmware and perform hardware-in-the-loop testing.

A configurable I2C device emulator is therefore useful as a hardware-in-the-loop testing tool. HIL methods are used because they provide repeatable, controlled test environments for embedded software and allow fault conditions to be exercised without changing the real system under test. For a firmware developer working with I2C peripherals, an emulator can shorten iteration time, improve repeatability, and make it easier to test both normal transactions and adverse cases before final hardware is fully available. MorphIIC is motivated by this gap. It aims to provide a simple, low-cost way to emulate I2C devices for repeatable firmware validation and hardware-in-the-loop testing.

3 Objective and Requirements

This section defines the minimum capabilities of MorphIIC to be considered successful. Based on the motivation stated in the introduction above, the project is aimed at providing a practical and configurable I2C device emulator for HIL testing. The objectives and requirements below establish the baseline functionality of the minimum viable product (MVP) and list some additional objectives for future development.

3.1 Essential Device Requirements

Before higher-level emulator features are useful, the device must first satisfy the baseline electrical and protocol expectations of a normal I2C peripheral. At minimum, MorphIIC must operate within the valid 7-bit I2C addressing model, respond correctly to address phases and acknowledgements, preserve a consistent register address space, and respect the timing constraints expected by standard-mode and fast-mode bus transactions. It must also present logic levels and bus behaviour that are electrically valid for integration with real embedded masters, including correct open-drain signalling, idle-high bus behaviour through pull-ups, and non-disruptive participation on the shared bus [2].

These requirements are foundational rather than optional features. If the emulator does not satisfy the essential I2C protocol and electrical constraints, then any higher-level register modelling would be misleading because failures could be caused by invalid bus behaviour rather than by the firmware under test. For this reason, protocol correctness, valid addressing behaviour, timing compliance, and electrically compatible signalling are treated as prerequisite conditions for the rest of the MorphIIC design.

3.2 Objectives

1. The system should allow developers to test firmware against an emulated peripheral when the real device is unavailable, inconvenient to use, or unsuitable for controlled testing.
2. The emulator should represent different peripherals through configuration rather than requiring custom firmware changes for each new device model.
3. The same inputs and test procedures should produce the same emulator behaviour across repeated runs so that results are reliable and debuggable.

4. The emulator should reproduce fixed register values and changing data patterns needed to mimic real sensor outputs or scripted test scenarios.
5. The initial implementation should focus on correct I2C behaviour, configuration driven modeling, and observability, while leaving more advanced behaviour and user experience features for future development.

3.3 Functional Requirements

Table 1: Functional Requirements for MorphIIC

ID	Requirement	Description	Motivation	Priority
FR1	Single-device I2C slave emulation	The system emulates at least one I2C slave device on a bus.	This is the core function required to satisfy O1. Without slave emulation the project cannot serve as a peripheral replacement for testing.	High
FR2	Configurable slave address	The emulated device uses a configurable I2C address.	Real peripherals use different bus addresses, so configurability is needed to make one platform usable across multiple test targets supporting O2.	High
FR3	Configurable register map	The device exposes a register map defined through a config file.	A configurable register model is key for representing different devices without firmware rewrites, supporting O2.	High
FR4	Read/Write transaction support	The system responds appropriately to master read and write operations with configured register data.	Correct transaction handling is necessary for the emulator to behave like a usable peripheral on an I2C bus, supporting O1 and O5.	High
FR5	Sequential multi-byte register access	The system supports multi-byte reads from consecutive register locations with an auto-increment feature.	Many register-mapped I2C devices provide this feature and master firmware often uses it, so this is needed for realistic emulation and avoiding firmware rewrites. Supporting O1 and O2.	High
FR6	Dynamic register data	The system supports read data that changes across successive reads for test scenarios.	Targets like sensors do not return constant values, so dynamic behaviour is needed to satisfy O4.	High
FR7	Repeatable	The same test will produce consistent behaviour across repeated runs.	Necessary for reliable debugging and regression testing, directly supporting O3.	High

ID	Requirement	Description	Motivation	Priority
FR8	Reconfiguration without firmware edits	New emulated device behaviours should be created by modifying configuration data without the need of making manual firmware changes.	The main mechanism by which MorphIIC avoids per-device implementations, directly supporting O2.	High
FR9	Device diagnostics	The device holds a log of incoming and outgoing transactions that can be accessed by developers.	Observability makes test failures easier to interpret and debug, supporting O3 and O5.	High
FR10	Host-streamed register data	The system supports runtime streaming of register data from a host so that dynamic datasets are not limited by onboard memory.	Test datasets may be too large to embed in firmware, so host streaming extends the usefulness of the platform while supporting O4.	Medium
FR11	Controlled NACK faults	The device is able to produce a NACK response to master on command or given certain conditions.	Fault injection improves the platform's usefulness for robustness testing, extending O1 beyond normal-case emulation.	Low
FR12	Configurable inter-register relations	The emulator supports basic functional relationships between registers and the data they hold.	Some devices cannot be represented by independent registers alone, so limited inter-register logic extends configurability in support of O2.	Low

Based on the aforementioned objectives and requirements, a possible solution might have the high-level system architecture depicted in figure 1 below.

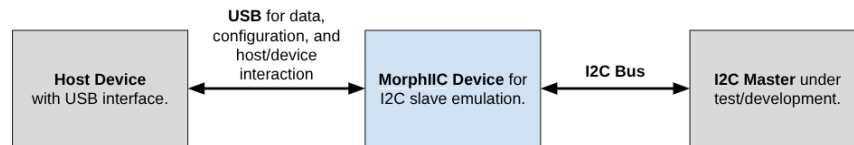


Figure 1: High-level system architecture of possible solution.

4 Design Decisions

4.1 Firmware Architecture

The firmware architecture needs to support the three concurrent responsibilities: I2C slave servicing, host interaction via the CLI, and USB-based streaming, all within a `no_std` embedded environment and with minimal register-level code for controlling microcontroller (MCU) peripherals. The two components that will impact how smoothly development goes are the microcontroller's hardware abstraction layer (HAL) and the concurrency framework. In the world of embedded Rust there are two main options for this: The first is making use of an independent HAL like `stm32g4xx-hal` or

`esp32-hal` with a HAL agnostic executor/concurrency framework, the most popular being RTIC [3]. The second option being use of the Embassy framework which is the pairing of one of Embassy’s HALs with the `embassy-executor` [4]. In practice, the most important evaluation criteria were integration effort, concurrency model ease of use, and how naturally the framework fit the communication-heavy structure of MorphIIC.

The project does not require any hard real-time guarantees, so a cooperative async model fits the project well, making use of Rust’s existing `async/await` framework. Embassy reduces implementation friction by providing both HAL and the executor in a single ecosystem, which avoids integration headaches from separate HAL and executors. Moreover, Embassy is one of the largest embedded Rust projects and for many microcontrollers have more complete HAL implementations. I2C slave configuration for microcontrollers is fairly niche, so Embassy HALs have a much higher chance of implementing the necessary abstractions to avoid register level writes. For these reasons, Embassy-HAL with `embassy-executor` was selected. The main tradeoff is reduced hardware flexibility. Relying on the Embassy framework limits MCU options to those supported by the small team of developers that develop and maintain the project. However, for the scope of MorphIIC, this limitation is acceptable as the framework supports the majority of popular microcontrollers (STM32s, RPxxxx, ESP32, NRF, etc.) [4].

4.2 Hardware Platform

The microcontroller platform selected for MorphIIC needs to be low-cost, USB-capable, practical for embedded Rust development, and most importantly, capable of reliable I2C slave behaviour. The candidate families considered are the RP2040, STM32 devices of C0, G4, F4 and F0 variants, and ESP32. The initial list of candidates is largely influenced by what is currently available to me, but my collection still covers the majority of popular MCUs in the embedded FOSS community. The candidates were evaluated not only on raw hardware capability, but also on practical development considerations like board availability, Rust ecosystem maturity, and the amount of low-level work required to achieve stable slave-mode behaviour.

STM32 devices are extremely popular in industry with very strong embedded Rust support, but compared to the RP2040 and ESP32 they are less common in inexpensive hobbyist-friendly development boards outside the official Nucleo line, which would make the device less accessible and flexible. This is evident in the product offerings of Adafruit Industries, a leading provider of open-source electronics, which include only two STM32-based development boards compared to over 20 options for the ESP32 and RP2040 [5]. ESP32 boards are widely available, but the lack of support for I2C slave behaviour in the Rust `esp-hal` would force a much more complex low-level implementation of the peripheral [6]. Thus, the RP2040 offers the best balance of inexpensive, accessible, and well-supported in the embedded Rust ecosystem with support for I2C slave operation in Embassy-HAL [7]. Another pro for the RP2040 is its unique Programmable Input Output (PIO) system. PIO pins are driven by programmable state-machines on separate hardware from the main Cortex-M0+ processor that allow for the user to create additional hardware interfaces, or even new types of interface in a robust, time-precise manner without bogging down the main processor [8].

A major tradeoff with the RP2040 is that it cannot cleanly issue a slave-side NACK for fault injection during normal operation, it requires a full peripheral clear and restart to go into a NACK issuing state [9]. Controlled NACK faults (FR11) were defined as a low-priority requirement, and so this limitation does not outweigh the numerous other practical advantages of the RP2040.

4.3 I2C Service Emulation Strategy

The emulation strategy needs to result in a device that behaves like a normal register-mapped I2C slave device from the master’s perspective. This includes correct handling of register pointers, sequential reads and writes, and standard bus-level signalling. Three approaches were considered: hardcoding device-specific per-register handlers directly into firmware, or using the MCU’s hardware I2C peripheral for bus signalling in combination with a generic-register model in firmware, and implementing the bus signalling using PIO with the same generic-register model in firmware.

A fully custom PIO-based bus handler would offer the most control and would allow for behaviour that the RP2040’s hardware peripheral does not support, but it would also require substantially more development time and verification effort because timing sensitive bus protocol details would need to be recreated in software. Hardcoded per-register handlers would be simple for a single device and would allow for easier implementation of inter-register relationships, but this shifts a lot of the complexity to model definition and the generation stage. This goes against MorphIIC’s goal of being easily reconfigurable. Using the hardware I2C peripheral for signalling (the `DW_apb_i2c` in the case of the RP2040), while keeping device behaviour in a generic register model addresses both concerns with the aforementioned options.

The peripheral handles all complex protocol specific timing and signalling, while the firmware focuses on register semantics and the model driven behaviour. This choice is the most practical and aligns best with the project objective of producing a reusable emulator rather than a one-off device mock. The main tradeoff is that the RP2040 peripheral does not allow for slave NACK in normal operation, as previously mentioned. Again, this tradeoff can be excused in the favour of prioritizing reliable normal case operation which is of higher priority for the MVP.

4.4 Configuration Format

The configuration format needs to be easy for users to edit, easily practicable by the generator to validate, and suitable for producing deterministic firmware tables at build time. The main options considered are JSON, YAML, TOML, and a custom runtime-editable data structure. They were evaluated on readability, familiarity, validation simplicity, and fit with the static generation workflow previously described. JSON was the strongest fit because of its simple structure, broad tooling support, straightforward validation workflow, and visually it’s well suited to represent a model with a small set of top-level device settings followed by a nested register table [10][11]. This makes it easy to understand from a syntax and visual level.

YAML is more flexible for humans, but that flexibility also makes it easier to introduce formatting ambiguity [12]. TOML is readable, but is less natural for representing larger repeated register entries than a JSON array of structured objects [12]. A runtime editable model was also considered, but it would add USB protocol complexity, increase the amount of mutable states in the system, and reduce determinism if the active device model can change during execution. Since one of MorphIIC’s top priorities is repeatability, a build time generated model is better suited for the project. For this reason, JSON with a firmware generation layer was selected. The tradeoff with this decision is updating the model requires rebuilding and reflashing the firmware onto the microcontroller. A side-by-side comparison of equivalent model snippets in all three formats is shown in Table 2.

A full JSON model example and the generated Rust output it produces are provided in Appendix 10.2.

Table 2: Equivalent MorphIIC Model Snippets Across JSON, TOML, and YAML

JSON	TOML	YAML
<pre>{ "device_name": "example", "i2c_address_7bit": 42, "default_fill": 255, "auto_increment": true, "i2c_internal_pullups": true, "i2c_respond_to_general_call": true, "registers": [{ "addr": 0, "default": 11, "access": "ro", "name": "REG1" }] }</pre>	<pre>device_name = "example" i2c_address_7bit = 42 default_fill = 255 auto_increment = true i2c_internal_pullups = true i2c_respond_to_general_call = true [[registers]] addr = 0 default = 11 access = "ro" name = "REG1"</pre>	<pre>device_name: example i2c_address_7bit: 42 default_fill: 255 auto_increment: true i2c_internal_pullups: true i2c_respond_to_general_call: true registers: - addr: 0 default: 11 access: ro name: REG1</pre>

4.5 Dynamic data strategy

The register data subsystem needs to support values that advance across successive reads, including long dynamic sequences derived from CSV data. The options considered were embedding the full series directly in firmware, streaming values continuously directly from the host at runtime, or using buffered host streams. They were evaluated based on memory limits, robustness, and their ability to make use of real gathered sensor data.

Fully embedded series are the simplest and most reliable option because all data is local to the MCU and there is no risk of host communication delay affecting reads. However, embedded series are limited by available flash and RAM. For smaller series this is acceptable, but for longer real-world datasets the approach isn't suitable. For example, a single accelerometer axis sampled at 1kHz for an hour would generate over 3MB of data, which is too large to store on most microcontrollers. Direct host streaming removes the length limit, but it makes the device too dependent on host timing and USB delivery which creates a high risk of underruns if the master reads faster than the host can supply data. This method also becomes much more complex as the number of host-streamed registers increases.

The middle ground is buffered host streaming where a subset of the data is stored on the MCU at any given time and the buffer absorbs timing variation between USB streaming protocol and any I2C reads. Considering all three options, a hybrid approach is taken. MorphIIC supports embedded data series for when maximum robustness with shorter data series is desired, and buffered host streams where unbounded data series are needed. This approach is more complex than either of the in-between solutions because it requires refill logic, underrun handling, and buffer accounting, but it best matches the project goals of supporting repeatable tests and more realistic long duration sensor playback.

5 System Overview

MorphIIC is an RP2040-based I2C device emulator designed to behave like a configurable register-mapped peripheral. Its behaviours are defined by a JSON device model, with optional CSV data sources for registers whose values need to change over time. The overall workflow is to describe the target device once in a model file, generate the corresponding firmware data structures at build time, then run that firmware on the RP2040 so that an external I2C master can interact with it as

though it were a real hardware peripheral.

At its core, MorphIIC emulates a single 7-bit I2C slave device with an 8-bit register address space. Each register can be assigned a default value and access permissions such as read-only or read-write, and the emulator supports standard pointer-based register access with optional auto-increment behaviour. This makes it suitable for reproducing the register semantics of many simple sensors and peripherals. In addition to fixed register values, MorphIIC also supports dynamic register data through CSV-backed read-only registers. These can either be embedded directly into the firmware at build time or streamed from a host at runtime over USB, allowing the emulator to reproduce both static device states and time-varying measurement data.

The system can be understood as a few separate subsystems working together. The I2C service is responsible for handling bus transactions, including register pointer updates, register reads and writes, and general-call behaviour. Beneath this the register file maintains the 256-byte register space, tracks which locations are writable, and manages read sequencing. For registers backed by dynamic CSV data, a stream manager provides values either from datasets embedded in firmware or from host supplied runtime streams. When data is streamed from a host to MorphIIC, data underruns, dropped samples, and most recently delivered values are tracked. In tandem, the USB subsystem exposes two separate CDC channels, one for a user-facing command-line interface (CLI), and another for the streaming protocol used to feed data series to registers with dynamic contents. A key part of the architecture is that the device model is validated and transformed before firmware is built. The model generator parses the JSON model describing the peripheral to be emulated, loads any referenced CSV files, checks for configuration errors, enforces memory, stream, and value limits, and lastly, generates the Rust source code containing the constant data structures referenced by the firmware. This keeps the runtime implementation simple, since most structural decisions about the emulated device are resolved ahead of time. On the host side, a feeder tool can connect over USB, negotiate stream configuration, pre-fill runtime buffers, and continue supplying new samples to any host-stream register as the device is used.

A complete worked example using one model across all stages (model JSON, CSV sources, generated firmware model tables, streamer output, and CLI output) is included in Appendix 10.2 and Appendix 10.3.

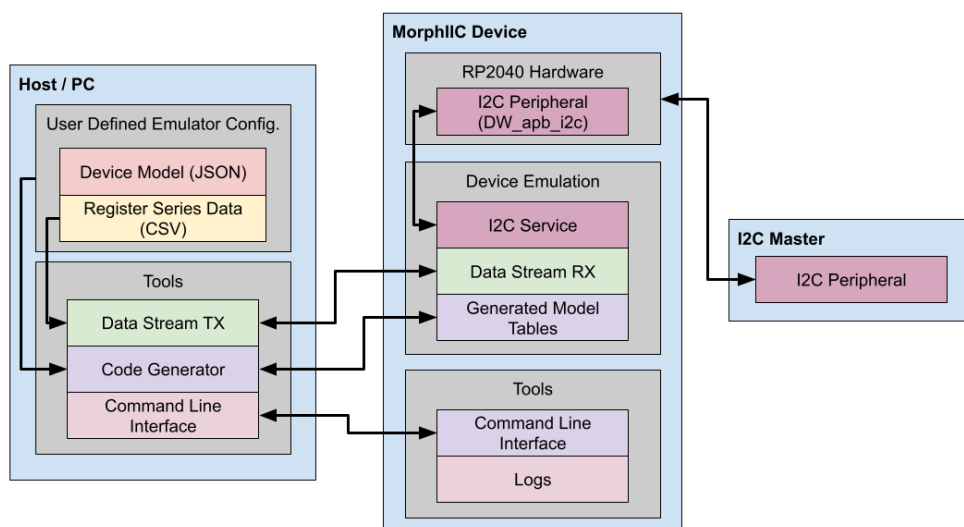


Figure 2: High-level host/device/master architecture.

To use MorphIIC, the user first defines the emulated device in a model file and optionally provides CSV sources for dynamic registers. That model is then converted into generated firmware data during the build process, after which the firmware is flashed to the RP2040. If the model includes host-streamed dynamic data registers, a host tool is started to feed runtime data over USB. Once running, the RP2040 presents itself to the external system as an I2C peripheral. During this time, the CLI channel can be used to inspect the device state, view or clear diagnostic logs, and reset the I2C interface.

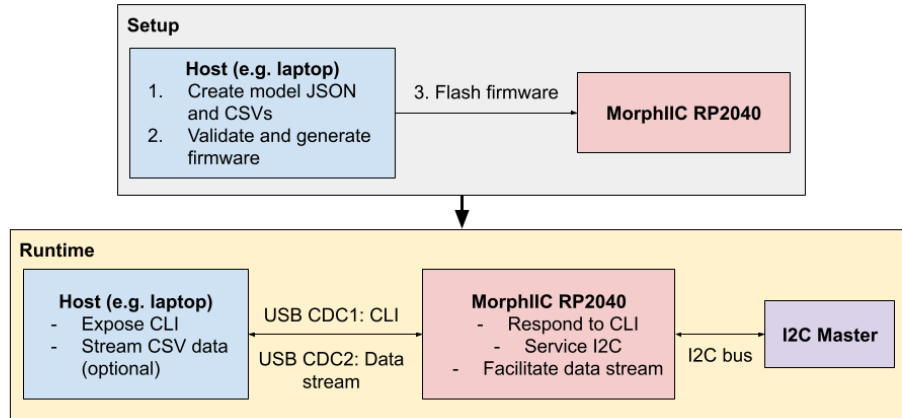


Figure 3: MorphIIC setup and runtime workflow.

6 Implementation

6.1 Hardware

During development, the Waveshare RP2040 Zero development board is used. It comes with a 2MB external flash IC installed. It can be powered and programmed via the USB-C connection which makes it extremely convenient for developers and MorphIIC users. There is also an on-board WS2812 RGB LED which can be used as a status LED controlled by GPIO pin 16. Throughout development, the I2C1 bus is used on GPIO pins 2 and 3 acting as SDA and SCL respectively.



Figure 4: Waveshare RP2040 Zero development board.

6.2 I2C Service

The I2C service is the core runtime loop. It waits asynchronously for commands from an I2C master (Write, WriteRead, Read, and GeneralCall) and translates them into register operations. For writes, the first byte is treated as a register pointer (as per I2C protocol standards) and all

following bytes are applied as payload. If the payload is several bytes, valid registers will be written to in sequential order if the auto-increment flag is set during model definition. For reads, the service serves data from the current pointer and if sequential read commands are with the auto-increment flag set, register values will be served in sequential address order. A reset signal path allows software triggered I2C peripheral reset from the CLI and general calls. In all cases, the event along with the current register pointer, payload, errors, and other info are logged to MorphIIC's embedded log. At a code level the I2C runtime is built around `i2c_service::task(...)` which multiplexes two async events:

1. `I2C_RESET_SIGNAL.wait()` for I2C peripheral resets issued by the host via CLI.
2. `slave.listen(&mut listen_buf)` for bus transactions using Embassy-HAL's abstractions.

This was achieved with `embassy_futures::select(...)` as a part of their utils for working with futures in `no_std` memory constrained environments.

Two service functions are primarily used to streamline the I2C service, `serve_read(...)` and `write_payload(...)`. The `write_payload(..., payload: &[u8])` function modifies the mocked registers by processing the provided payload. For each byte, it verifies if the target register is configured for write access; if so, it updates the value in the register table. Additionally, if auto-increment is enabled in the device model, the function advances the register pointer to the next address.

`serve_read(...)` repeatedly pulls data from the register tables until the Embassy I2C read status is no longer `NeedMoreBytes`, it places data bytes one at a time into the I2C peripheral FIFO buffer and sends them using Embassy's `respond_to_read()`. Because reads can advance stream states and register pointers, it tracks a per-byte `ReadEffect` and if the master stops early stream position or register pointers are rolled back.

For I2C transaction behaviour:

- `Write(len)`: Takes the first byte of the incoming message which becomes the register pointer and remaining bytes are passed to `RegisterFile::write_payload` to update embedded register table data.
- `WriteRead(len)`: Takes the first byte of the incoming message which becomes the register pointer and if additional bytes are provided, they are written. `Serve_read` is then called from the newly updated register pointer moving the requested data into the I2C peripherals FIFO buffer where it's then used in the read response on the I2C bus.
- `Read`: `serve_read()` is called from the current register pointer, if auto-increment is enabled in configuration, it will move the pointer forward by one address.
- `GeneralCall(len)`: Takes the first byte of the payload as the general-call command (if present). If general-call response is enabled and the command is `0x06`, MorphIIC resets the I2C slave and restores non-CSV registers to defaults (including register pointer reset). In all cases, the general-call transaction is logged.

6.3 Device Model

The device model is driven by a JSON file that defines the identity, addressing, register defaults, register access, and optional CSV backed behaviour. The generation script runs during build, validates the constraints listed below, and creates static Rust tables used by the firmware:

- The address is a valid 7-bit I2C address `0x08..=0x77`.
- The register count is ≤ 256 .
- All register addresses are ≤ 255 .
- There are no duplicate register addresses.
- Model shape is valid and no fields are missing or unknown.
- CSV backed registers are configured as read-only and provided.
- CSV file must load, produce at least one sample, and all samples must be within `u8`.
- Host-stream register count does not exceed `MAX_HOST_STREAM_REGS = 9`.
- Total embedded data series does not exceed a budget of 32KiB.

These checks catch issues with the user provided model early and fail with explicit errors instead of at runtime on the RP2040.

Appendix 10.2 includes a concrete model file, CSV inputs, and the generated `model.rs` output from that same model.

CSV backed registers support both hex and decimal formatting, expect data points to be separated by row, ignore comments and blank lines, and enforce the unsigned 8-bit data range. Embedded data series have their data sequence generated into a constant array in the firmware model but before this, the total memory number of data points is checked to ensure it's under the 32KiB budget.

The generated model (`model.rs`) is entirely made up of constant data structures. `REGISTERS` holds the `RegisterSpec` containing address, default value, access rules, and name for each user defined register. `CSV_REGISTERS` holds the `CsvRegisterSpec` containing address, `CsvMode` (either `Embedded` or `HostStream`), and data series if `Embedded`. `CSV_INDEX_BY_ADDR` provides a lookup table for the index of the register in `CSV_REGISTERS`. `HOST_STREAM_INDEX_BY_ADDR` provides a lookup table for the stream in `HOST_STREAM_ADDRS` which holds the register addresses of host streamed registers.

Lastly, the data in `model.rs` is loaded into a `RegisterFile` struct with a register array [`u8; REG_COUNT`], read/write rule lookup [`bool; REG_COUNT`], and the current register pointer to be used at runtime. `RegisterFile` exposes several methods to interact with the register fields such as `read_into()` and `write_payload()`.

6.4 Dynamic Register Data Series

Dynamic register data series are implemented through a stream manager with two modes:

1. Embedded: values are compiled into firmware and returned in a looping sequence per register.
2. Host Stream: values are pushed at runtime into per-register circular buffers and consumed on reads.

For host streams, the system tracks buffer level, free space, underruns, drops, and last value. If a stream underruns, the read falls back to the last-known value (or register fallback if none exists yet). Side effects from the read are tracked so they can be reversed when necessary (for partial or rolled-back reads). This subsystem is what allows MorphIIC to go beyond static emulation and provide dynamic live-sensor-like behaviour.

For data transfer and upkeep, a custom protocol was written:

1. **Negotiation (host to device stream discovery):** The host tool starts by sending `HELLO_REQ` (0x01), and firmware replies with `HELLO_RESP` (0x81), protocol version, stream count, and per-stream descriptors [`stream_id`, `register_addr`, `capacity_le_u16`]. The host validates version and descriptor length, then maps each `host_stream` register from the model to a returned stream descriptor before any data transfer begins.
2. **Transfer protocol:** Frames use a 3-byte header [`opcode` + `payload_len_le_u16`], followed by payload. For feeding, the host sends `FEED` (0x02) with payload [`stream_id`, `data0`, `data1`, ...]. Firmware appends accepted bytes into the target FIFO buffer and returns `FEED_ACK` (0x82) with [`stream_id`, `accepted_le_u16`, `free_le_u16`]. If buffer space is insufficient, excess bytes are counted as drops on the RP2040 side and the host rewinds its sample cursor so dropped bytes do not go unread and the series is preserved.
3. **Upkeep:** After negotiation, the host pre-fills each stream to 75% capacity, then runs an upkeep loop that every `PROBE_INTERVAL` sends a zero-data feed to refresh free-space knowledge from the RP2040 ACK. If the data buffer is below 50% full, the host pushes another chunk (default 48 bytes).

A representative `csv_streamer` runtime session for this workflow is provided in Appendix 10.3.

6.5 Device logs

The device log system is built for embedded constraints with no heap allocation, bounded memory use, and low overhead during I2C handling. Logs are stored in a fixed-size ring buffer with `RING_CAPACITY = 128` and fixed log size. Each log captures the event type (Write, Read, General-Call, Error, etc.), register pointer, the transaction length, status, error code, and a 15 byte preview of data. Each log has a sequence number with a `u32` wrap limit. The ring buffer is guarded by a `critical_section::Mutex` so log modifications are atomic with respect to the other tasks.

Logs are formatted using a `format_event_line()` producing a fixed size `String` for all log variations. These formatted logs are then available to the user using the `dump` command via the CLI.

This approach works well with the embedded constraints of the RP2040 and keeps logging deterministic while still preserving enough context to debug transaction behaviours and protocol errors.

6.6 CLI

The CLI runs on a dedicated USB CDC channel and provides operational control and observability to the user. The dedicated task loops waiting for a connection. Once connected, a startup banner with available commands is sent, and the task continuously reads incoming USB packets. Any read error breaks the loop and the task returns to the connection waiting state so reconnection is automatic but error handling is minimal.

Commands are case-insensitive, terminated by new line character or carriage return, and decoded as UTF-8:

- `Dump`: dumps I2C transaction log ring.
- `Clear`: clears transaction log ring.
- `Reset_i2c`: resets RP2040 I2C peripheral.
- `Stream_status`: show host stream buffer status.

- Help: shows available commands.

Unknown commands are responded to with an unknown command error. This gives a debug interface without requiring reflashing firmware or external debugger for basic runtime inspections.

A sample CLI session showing `help`, `dump`, `stream_status`, and `clear` is included in Appendix 10.3.

7 Validation and Testing

7.1 Model Testing

Model-side testing uses a combination of unit, property, and integration tests. Unit tests confirm parser, validator, and generator behaviour for boundary, valid, and malformed cases. Property-based tests supplement this by generating randomized valid and invalid inputs to exercise edge conditions beyond hand-written models. Integration tests verify end-to-end behaviour through `parse_model` and `generate_from_paths` against real model JSON and CSV files.

Together, these tests confirm parser correctness, generator determinism, and expected failures under invalid input conditions. Host integration outcomes are summarized in Table 3, and host unit/property outcomes are summarized in Table 4.

Table 3: Host Integration Tests

Test name	What it does	Targeted case	Result
<code>fixture_valid_model_parses_and_generates</code>	Parses a known-good mixed model, resolves CSV paths, and runs code generation.	Happy-path end-to-end flow for a valid model (parse and generate).	Generated <code>/tmp/model_valid.rs</code> .
<code>invalid_duplicate_addr.json</code> (integration error case)	Loads the invalid model and confirms generation fails with duplicate-address validation.	Validation rejects duplicate register addresses.	Error generating <code>/tmp/model_invalid_duplicate.rs</code> from <code>tools/gen_model/tests/models/invalid_duplicate_addr.json</code> . Caused by: duplicate register addr 2.
<code>invalid_unknown_field.json</code> (integration error case)	Loads a model with an unexpected key and confirms parse failure includes field context.	Strict JSON schema enforcement (unknown top-level field).	Error generating <code>/tmp/model_invalid_unknown.rs</code> from <code>tools/gen_model/tests/models/invalid_unknown_field.json</code> . Caused by: parsing model JSON; unknown field <code>unexpected</code> ; expected one of <code>device_name</code> , <code>i2c_address_7bit</code> , <code>i2c_internal_pullups</code> , <code>i2c_respond_to_general_call</code> , <code>default_fill</code> , <code>auto_increment</code> , <code>registers</code> at line 9 column 14.
<code>invalid_enum.json</code> (integration error case)	Loads a model with an invalid access enum value and confirms parse failure.	Enum validation for register access values (<code>ro/rw</code> only).	Error generating <code>/tmp/model_invalid_enum.rs</code> from <code>tools/gen_model/tests/models/invalid_enum.json</code> . Caused by: parsing model JSON; unknown variant <code>write_only</code> , expected <code>ro</code> or <code>rw</code> at line 12 column 28.
<code>invalid_csv_rw.json</code> (integration error case)	Loads a model where a CSV-backed register is RW and confirms validation failure.	Constraint that CSV-backed registers must be read-only.	Error generating <code>/tmp/model_invalid_csv_rw.rs</code> from <code>tools/gen_model/tests/models/invalid_csv_rw.json</code> . Caused by: register addr 4 uses CSV but is not read-only.

Test name	What it does	Targeted case	Result
model_integ_tests suite	Executes integration tests over valid/invalid models and CSV loading behaviour.	Integration-level coverage for parse/validate/generate and model CSV reads.	running 3 tests ... test result: ok. 3 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.00s.

Table 4: Host Unit and Property Tests

Test name	What it does	Targeted case	Result
parse_model_accepts_boundary_variants	Parses two valid extremes: a minimal one-register model and a larger mixed model with host stream, embedded CSV, and RW registers.	Acceptance at boundary configurations (lowest/highest I2C limits, maximum host-stream count, mixed register types).	PASS (cargo test -p gen_model -lib).
parse_model_accepts_sparse_register_addresses	Parses a model with non-contiguous register addresses and mixed RO/RW access.	Sparse address-map acceptance without requiring contiguous register space.	PASS.
parse_model_rejects_invalid_shape_and_constraints	Exercises a set of malformed/invalid models and verifies expected validation or parse errors.	Address bounds/reserved ranges, duplicate/out-of-range registers, CSV constraints, and schema/enum errors.	PASS. Expected errors include: bad_i2c_address_out_of_7bit_range: i2c_address_7bit must be in 0x08..=0x77. out_of_range_register: register addr 256 is outside 8-bit address space (0..=255). duplicate_address: duplicate register addr 2. csv_on_rw: register addr 2 uses CSV but is not read-only. empty_csv_path: register addr 2 has empty csv.path. too_many_host_stream_registers: host_stream register count 10 exceeds limit 9. unknown_field: parsing model JSON failed on unknown field unknown_field. missing_field: missing field i2c_internal_pullups. invalid_enum: unknown variant write_only, expected ro or rw.
parse_csv_samples_parses_expected_formats	Parses CSV samples with supported formats and ignores comments, blank lines, and extra columns.	Positive CSV parser acceptance rules (decimal/hex variants and first-column-only handling).	PASS (parsed output matched expected byte vectors).
parse_csv_samples_rejects_invalid_content	Feeds malformed CSV content and checks function-level errors for each invalid pattern.	CSV parser failure handling for invalid tokens, out-of-range values, and empty usable input.	PASS. Expected function-level errors include: invalid decimal byte abc at bad.csv:1. invalid hex byte 0xGG at bad.csv:1. value 256 out of range at bad.csv:1 (must be 0..255). bad.csv does not contain any usable CSV samples.
generate_fw_model_sorts_and_builds_indices	Generates firmware model code from a mixed register set and asserts deterministic ordering/table/index outputs.	Codegen invariants: sorted registers, host-stream maps, embedded symbol emission, and config constants.	PASS.
generate_from_paths_enforces_embedded_budget	Generates from paths with oversized embedded CSV payload and verifies hard budget rejection.	Embedded CSV cumulative budget guardrail (32768 bytes).	PASS. Expected error: embedded CSV payload exceeds budget at 32769 bytes total (budget 32768, over by 1); per-register embedded sizes report 0x20:32769B.

Test name	What it does	Targeted case	Result
prop_random_valid_models_parse (proptest)	Randomly builds valid models over address, default, flag, and register-set combinations and asserts parser acceptance.	Property-based robustness for valid-model parsing across broad input combinations.	PASS.
prop_targeted_invalid_duplicate_rejected (proptest)	Randomly generates duplicate-address models and asserts consistent rejection.	Property-based enforcement of duplicate register-address detection.	PASS (duplicate-address rejection consistently observed).
prop_csv_round_trip_randomized (proptest)	Randomly generates CSV lines (hex, decimal, comment, and extra-column variants) and verifies parsed bytes match expected sequence.	Property-based round-trip correctness of CSV parsing semantics.	PASS.

7.2 Device Testing

To perform device testing, two maximum-value device models are emulated (9 host-streamed registers, 32768 bytes of embedded data, and one of each static register type): one with auto-increment enabled and one without. An STM32G474 Nucleo development board is used as the I2C master. Device testing is split into deterministic functional validation and long-run stress validation at 100 kHz and 400 kHz I2C bus rates.

Functional testing is executed across four configuration profiles to isolate behavior by design: streamer inactive/active combined with auto-increment disabled/enabled. This ensures host-stream behavior is validated independently for each pointer mode. Stress testing is split into two profiles (auto-increment off and on), with each profile using an S05 variant that matches its access pattern: repeated single-register reads for no-auto-increment, and repeated 9-register ring bursts for auto-increment.

Functional outcomes are summarized in Table 5, and stress outcomes are summarized in Table 6. For reproducibility context, Appendix 10.2 and Appendix 10.3 provide one complete example artifact set (model inputs, generated firmware tables, streamer output, and CLI output) captured from the same device model.

Raw STM32G474 master logs for each functional and stress profile are provided in Appendix 10.4. A diagram of the hardware/software test setup is provided in Fig. 5 in Appendix 10.4.

Table 5: Device Functional Tests

Test name	What it does	Targeted case	Result
F01_STATIC_DEFAULTS	Reads static RO/RW registers and checks model default values.	Incorrect static register initialization/default emission.	PASS at 100 kHz and 400 kHz in all 4 functional profiles (ai_on/off x stream_on/off).
F02_RW_RO_MULTIBYTE	Performs multibyte write and verifies RW mutability + RO immutability, with expected RW outcome dependent on auto-increment mode.	Access-permission bugs, write-path bugs, and auto-increment write side-effects.	PASS at both rates in all 4 functional profiles.
F03_AUTO_INCREMENT_FLOW	Verifies pointer/read behavior explicitly for auto_increment=true vs false .	Pointer progression bugs and wrong auto-increment mode behavior.	PASS at both rates in all 4 functional profiles.

Test name	What it does	Targeted case	Result
F04_GENERAL_CALL_RESET	Writes to rw byte, then issues general-call reset and checks post-reset values to confirm register values are restored to defaults.	Verifies general call reset behaviour.	PASS at both rates in all 4 functional profiles.
F05_EMBEDDED_SEQ	Repeatedly reads each embedded register directly and checks sequence consistency.	Embedded CSV cursor/sequence logic errors.	PASS at both rates in all 4 functional profiles.
F06_HOST_EXPECT_INACTIVE	In stream-off profiles, validates host-stream fallback behavior and absence of feeder-seed activity.	False host-stream activity when feeder must be inactive.	Executed in *_stream_off profiles only; skipped in *_stream_on . PASS at 100 kHz and 400 kHz for both ai_on_stream_off and ai_off_stream_off .
F07_HOST_EXPECT_ACTIVE	In stream-on profiles, validates active feeder continuity across all host streams.	Feeder continuity/mapping issues and partial stream activity.	Executed in *_stream_on profiles only; skipped in *_stream_off . PASS at 100 kHz and 400 kHz for both ai_on_stream_on and ai_off_stream_on .

Table 6: Device Stress Tests

Test name	What it does	Targeted case	Result
S01_MIXED_LOOP	Long mixed loop of RO reads, RW writes/reads, and embedded sequence checks.	State corruption, intermittent I2C faults, embedded sequence regressions under mixed load.	PASS at 100 kHz and 400 kHz in both stress_ai_off and stress_ai_on logs.
S02_BURST_LOOP	High-rate burst/static reads plus repeated embedded per-register sequence checks.	Burst-transaction instability and embedded consistency under sustained load.	PASS at 100 kHz and 400 kHz in both stress profiles.
S03_HOST_SOAK	Long host-stream soak with feeder active; checks activity/continuity metrics.	Host-stream starvation, low activity, feeder continuity issues.	PASS at 100 kHz and 400 kHz in both stress profiles.
S04_STOP_RESUME	Manual feeder interruption/recovery: explicit OFF prompt, enforced 2049-read drain before plateau check, then ON prompt and resume check.	Recovery behavior after feed interruption and post-drain plateau/resume behavior.	At 100 kHz: PASS in both stress profiles. This test is redundant at 400 kHz and therefore skipped in both profiles.
S05_SINGLE_REG_SEQ_UNDERRUN_AI_OFF	(auto_increment=false) sequential single-register host-stream reads with tunable read count/delay; logs repeat and timing metrics (avg/min/max inter-read us).	Underrun on single-stream depletion path and timing characterization.	PASS at 100 kHz and 400 kHz in stress_ai_off ; observed equal-adjacent-pairs = 0 (no inferred underrun), with sequential read intervals: 100 kHz 843/762/946 us, 400 kHz 575/549/701 us.

Test name	What it does	Targeted case	Result
S05_RING_SEQ_UNDERRUN_AI_ON	(<code>auto_increment=true</code>) repeated 9-byte ring reads (<code>0x10..0x18</code>) with pointer set to first host register each burst; per-stream repeat stats + timing metrics.	Underrun behavior in auto-increment ring-read path across all host streams plus timing characterization.	PASS at 100 kHz and 400 kHz in <code>stress_ai_on</code> ; observed equal-adjacent-pairs = 0 on all host streams (no inferred underrun), with burst intervals: 100 kHz 2030/1983/2166 us, 400 kHz 1273/1220/1434 us.

8 Discussion

8.1 Testing Summary

The validation process covered both model-generation tooling and on-device runtime behaviour. On the host side, unit, property, and integration tests exercised parser, validator, CSV handling, and code-generation paths for both valid boundary cases and malformed input cases (Tables 3 and 4). All reported tests passed. On the device side, functional tests (F01–F07) and stress tests (S01–S05) passed across both 100 kHz and 400 kHz configurations, with only the intentionally redundant S04 case skipped at 400 kHz (Tables 5 and 6).

Taken together, these results provide evidence that MorphIIC remains correct across the full workflow: from user model definition, through model generation and firmware build, to sustained I2C transaction handling on real hardware.

8.2 Model Validation and Developer Usability

A key practical outcome is that model validation catches most configuration issues at build time rather than at runtime on the RP2040. Validation checks for invalid 7-bit addresses, duplicate or out-of-range register definitions, CSV-backed access-rule violations, host-stream-count limits, and embedded-data memory budget violations. The test outcomes show these failures are surfaced with explicit and actionable error messages.

This design makes it easier for developers to create correct models quickly. Instead of debugging opaque embedded failures after flashing firmware, users receive direct feedback while writing model files. In practice, this lowers onboarding friction and shortens the edit-build-test cycle when introducing new devices.

8.3 Dynamic Throughput and Underrun Behaviour

The dynamic-register stress results are especially encouraging. In both S05 variants, no underrun was inferred under high-throughput operation, with `equal_adjacent_pairs=0` reported at both 100 kHz and 400 kHz. In single-register sequential mode at 400 kHz, the measured average inter-read interval was approximately 575 μ s (about 1.7 kSamples/s). In 9-register ring mode at 400 kHz, the average burst interval was approximately 1273 μ s, corresponding to roughly 0.8 kSamples/s per register (around 7.1 kSamples/s aggregate across the ring).

Reported sensor update rates in literature span several orders of magnitude depending on application, from environmental monitoring at seconds-to-minutes intervals, to GNSS tracking at 1-10 Hz, to wearable inertial tracking commonly near 100 Hz, and up to hundreds of hertz or kilohertz for some inertial and biomedical sensing tasks [13, 14, 15, 16, 17]. Under the host-stream configuration evaluated here, the measured throughput therefore appears most appropriate for lower-rate

environmental, status, and moderate-rate tracking emulation, particularly when multiple registers are being cycled. For single-register dynamic reads, however, the achievable polling rate is substantially higher, making the platform better suited to faster single-channel emulation scenarios than the multi-register ring results alone might suggest.

8.4 Limitations and Future Improvements

Despite the positive results, several limitations remain. First, explicit NACK control is not yet implemented. Many real peripherals NACK in state-dependent situations (for example, busy windows, invalid accesses, or startup delays), and being able to script this behaviour would improve driver testing.

Second, inter-register interaction semantics are currently limited. Real devices often have coupled behaviour where writing one control register affects status flags, output scaling, or data-ready bits in other registers. Adding a modelled interaction layer (for example, trigger-action rules on read/write/reset events) would substantially improve realism.

Additional future work includes expanding master testing beyond the current setup, adding automated latency/jitter characterization to validation, and supporting YAML as an alternative model format while preserving the same validation guarantees.

9 Conclusion

This report presented MorphIIC, a model-driven I2C peripheral emulation framework built around the RP2040. The system combines generated register-model data, a deterministic I2C service loop, and dynamic register streaming so that firmware teams can test against realistic register-level behaviour while keeping the same bus-facing integration path used with physical devices.

The validation results indicate that this approach is both practical and robust. Model-side tests show strong parser, validator, and generator correctness, while device-side functional and stress tests demonstrate stable behaviour at 100 kHz and 400 kHz under mixed static and dynamic workloads. Importantly, dynamic host-stream tests reported no inferred underrun during the evaluated high-throughput scenarios, supporting MorphIIC’s suitability for many moderate-rate embedded sensing emulation tasks.

Overall, MorphIIC reduces iteration cost by moving configuration errors to build time and providing repeatable runtime behaviour on hardware. The most valuable next steps are programmable NACK behaviour and richer inter-register interaction rules, which would improve realism for driver edge-case testing and bring the platform closer to full behavioural emulation of complex peripherals.

References

- [1] J. Jarboe, “Solving automated testing challenges for embedded systems devsecops,” Wind River Blog, Feb. 2024, [Online; accessed Apr. 4, 2026]. [Online]. Available: <https://www.windriver.com/blog/Solving-Automated-Testing-Challenges-for-Embedded-Systems-DevSecOps>
- [2] J. Wu, “A basic guide to i2c,” Texas Instruments, Tech. Rep. SBAA565, Nov. 2022, [Online; accessed Apr. 4, 2026]. [Online]. Available: <https://www.ti.com/lit/an/sbaa565/sbaa565.pdf>
- [3] Rust on Embedded Devices Working Group *et al.*, “The embedded rust book.” [Online]. Available: <https://docs.rust-embedded.org/book/>
- [4] embassy-rs contributors, “embassy,” GitHub repository, 2026, [Online; accessed Apr. 4, 2026]. [Online]. Available: <https://github.com/embassy-rs/embassy>
- [5] Adafruit Industries, “Adafruit industries: Unique & fun diy electronics and kits.” [Online]. Available: <https://www.adafruit.com/>
- [6] esp-rs contributors, “esp-hal,” GitHub repository, 2026. [Online]. Available: <https://github.com/esp-rs/esp-hal>
- [7] embassy-rs contributors, “embassy-rp,” GitHub repository, 2026. [Online]. Available: <https://github.com/embassy-rs/embassy/tree/main/embassy-rp>
- [8] A. Bate, “What is pio?” Raspberry Pi News, Mar. 2021, [Online; accessed Apr. 4, 2026]. [Online]. Available: <https://www.raspberrypi.com/news/what-is-pio/>
- [9] Raspberry Pi Ltd, *RP2040 Datasheet*, Feb. 2025, section 4.3: I2C, pp. 440–464, build-date 2025-02-20. [Online]. Available: <https://pip-assets.raspberrypi.com/categories/814-rp2040/documents/RP-008371-DS-1-rp2040-datasheet.pdf?disposition=inline>
- [10] T. Bray, “The javascript object notation (json) data interchange format,” Internet Engineering Task Force, RFC 8259, Dec. 2017. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc8259.html>
- [11] JSON Schema. (2026) What is json schema? [Online]. Available: <https://json-schema.org/overview/what-is-jsonschema>
- [12] YAML Language Development Team, “YAML Ain’t Markup Language (YAML™) Version 1.2.1,” 2009, specification. [Online]. Available: <https://yaml.org/spec/1.2.1/>
- [13] G. Provolo, C. Brandolese, M. Grotto, A. Marinucci, N. Fossati, O. Ferrari, E. Beretta, and E. Riva, “An internet of things framework for monitoring environmental conditions in livestock housing to improve animal welfare and assess environmental impact,” *Animals (Basel)*, vol. 15, no. 5, p. 644, Feb. 2025.
- [14] V. Gikas and H. Perakis, “Rigorous performance evaluation of smartphone GNSS/IMU sensors for ITS applications,” *Sensors (Basel)*, vol. 16, no. 8, p. 1240, Aug. 2016.
- [15] L. Zhou, E. Fischer, C. Tunca, C. M. Brahms, C. Ersoy, U. Granacher, and B. Arnrich, “How we found our IMU: Guidelines to IMU selection and a comparison of seven IMUs for pervasive healthcare applications,” *Sensors (Basel)*, vol. 20, no. 15, p. 4090, Jul. 2020.

- [16] N. Parnian and F. Golnaraghi, “Integration of a multi-camera vision system and strapdown inertial navigation system (SDINS) with a modified kalman filter,” *Sensors (Basel)*, vol. 10, no. 6, pp. 5378–5394, May 2010.
- [17] L. Wang, S. Tian, and R. Zhu, “A new method of continuous blood pressure monitoring using multichannel sensing signals on the wrist,” *Microsyst. Nanoeng.*, vol. 9, no. 1, p. 117, Sep. 2023.

10 Appendix

10.1 Appendix A: Personal Motivation

As an embedded system hobbyist and soon to be professional, I can think of many scenarios where MorphIIC would be extremely convenient during development. The first case is when I was working on a TinyML project that ran a custom MLP gesture recognition model on an STM32 reading IMU data via I2C. Having a way to feed exact repeatable sensor data to my system would have been extremely convenient during the development and debugging process of the model. The second case came up during capstone; we were considering integrating an IMU into our robotic hand and I could have started development on the driver and testing with hardware 10 days earlier without having to wait for shipping.

I also find the name very clever. *Morphic* as a suffix conveys the meaning “having the shape, form, or structure”. For example the word *zoomorphic* with the definition “having or representing animal forms or gods of animal form”. Add an extra “T” to morphic and you have a name that is phonetically the same as the morphic but also has the protocol used in the device IIC. On top of that, the device “morphs” or takes the structure of whatever device you are trying to emulate. In summary, I find the name satisfying.

10.2 Appendix B: Worked Example Model Artifacts

This appendix collects one consistent end-to-end example used throughout the report. The JSON model, CSV inputs, and generated firmware tables below all correspond to the same simple MorphIIC device configuration.

The generator command used for this example is: `cargorun-pgen_model--models/device_model.jsonfirmware/src/model.rs`

10.2.1 B.1 Device model JSON

Listing 1: Example model file.

```
{
  "device_name": "morphIIC",
  "i2c_address_7bit": 66,
  "default_fill": 255,
  "auto_increment": true,
  "i2c_internal_pullups": true,
  "i2c_respond_to_general_call": true,
  "registers": [
    {
      "addr": 0,
      "default": 18,
      "access": "ro",
      "name": "REG1"
    },
    {
      "addr": 1,
      "default": 52,
      "access": "ro",
      "name": "REG2"
    },
    {
      "addr": 16,
```

```

        "default": 0,
        "access": "rw",
        "name": "REG3"
    },
    {
        "addr": 17,
        "default": 0,
        "access": "rw",
        "name": "REG4"
    },
    {
        "addr": 32,
        "default": 160,
        "access": "ro",
        "name": "REG5_EMBEDDED",
        "csv": {
            "path": "csv/reg5_embedded.csv",
            "mode": "embedded"
        }
    },
    {
        "addr": 33,
        "default": 161,
        "access": "ro",
        "name": "REG6_HOST_STREAM",
        "csv": {
            "path": "csv/reg6_host.csv",
            "mode": "host_stream"
        }
    }
]
}

```

10.2.2 B.2 Embedded CSV source

Listing 2: Embedded dynamic data source.

```

# Embedded values
0xA0
0xA1
0xA2
0xA3
0xA4
255

```

10.2.3 B.3 Host-stream CSV source

Listing 3: Host-stream dynamic data source.

```

# Host values
10
20
30
40
50

```

```
60
70
80
90
100
```

10.2.4 B.4 Generated firmware model output

Listing 4: Generated `model.rs` output for the example model.

```
// Generated by tools/gen_model. Do not edit manually.

#![allow(dead_code)]

pub const DEVICE_NAME: &str = "morphIIC";
pub const I2C_ADDRESS_7BIT: u8 = 0x42;
pub const I2C_INTERNAL_PULLUPS: bool = true;
pub const I2C_RESPOND_TO_GENERAL_CALL: bool = true;
pub const REG_COUNT: usize = 256;
pub const DEFAULT_FILL: u8 = 0xFF;
pub const AUTO_INCREMENT: bool = true;
pub const MAX_HOST_STREAM_REGS: usize = 9;
pub const HOST_STREAM_BUFFER_CAPACITY: usize = 2048;
pub const EMBEDDED_CSV_BUDGET_BYTES: usize = 32768;
pub const EMBEDDED_CSV_TOTAL_BYTES: usize = 6;

#[derive(Copy, Clone, Debug, Eq, PartialEq)]
pub enum Access {
    Ro,
    Rw,
}

#[derive(Copy, Clone, Debug, Eq, PartialEq)]
pub enum CsvMode {
    Embedded,
    HostStream,
}

#[derive(Copy, Clone, Debug, Eq, PartialEq)]
pub struct RegisterSpec {
    pub addr: u8,
    pub default: u8,
    pub access: Access,
    pub name: &'static str,
}

#[derive(Copy, Clone, Debug, Eq, PartialEq)]
pub struct CsvRegisterSpec {
    pub addr: u8,
    pub mode: CsvMode,
    pub data: &'static [u8],
}

pub const REGISTERS: &[RegisterSpec] = &[
    RegisterSpec { addr: 0x00, default: 0x12, access: Access::Ro, name: "REG1" },
    RegisterSpec { addr: 0x01, default: 0x34, access: Access::Ro, name: "REG2" },
    RegisterSpec { addr: 0x10, default: 0x00, access: Access::Rw, name: "REG3" },
]
```


10.3 Appendix C: Runtime Session Outputs

This appendix contains runtime captures produced from the same example model after flashing firmware and running the host tools.

The host streamer command for this capture is:

```
cargo run -p csv_streamer -- <serial_port> models/device_model.json
```

10.3.1 C.1 Host streamer output

Listing 5: Representative csv_streamer output.

```
csv_streamer attached to /dev/ttyACM1 (1 stream(s) active)
  id=0 addr=0x21 capacity=2048 samples=10
--- csv_streamer debug ---
  id=0 addr=0x21 fill=1584/2048 free=464 feed_calls=1 probe_calls=20 accepted_total=48 cursor=8
--- csv_streamer debug ---
  id=0 addr=0x21 fill=1584/2048 free=464 feed_calls=1 probe_calls=40 accepted_total=48 cursor=8
--- csv_streamer debug ---
  id=0 addr=0x21 fill=1584/2048 free=464 feed_calls=1 probe_calls=60 accepted_total=48 cursor=8
--- csv_streamer debug ---
  id=0 addr=0x21 fill=1584/2048 free=464 feed_calls=1 probe_calls=80 accepted_total=48 cursor=8
--- csv_streamer debug ---
  id=0 addr=0x21 fill=1584/2048 free=464 feed_calls=1 probe_calls=100 accepted_total=48 cursor=8
```

10.3.2 C.2 CLI interaction output

Listing 6: Representative CLI interaction and device log output.

```
---- Sent utf8 encoded message: "help\r\n" ----
Commands:
  dump          -> dump I2C transaction log ring
  reset_i2c     -> reset RP2040 I2C peripheral
  clear         -> clear transaction ring
  stream_status -> show host stream buffer status
---- Sent utf8 encoded message: "dump\r\n" ----
--- log dump begin ---
#00001 WRRD ptr=0x00 len=3 st=0 err=0x00000000 data=12 34 FF
#00002 WRITE ptr=0x10 len=1 st=1 err=0x00000000 data=33
#00003 WRRD ptr=0x20 len=3 st=0 err=0x00000000 data=A1 14 FF
#00004 GCALL ptr=0x23 len=1 st=0 err=0x00000000 data=60
#00005 WRRD ptr=0x00 len=3 st=0 err=0x00000000 data=12 34 FF
#00006 WRRD ptr=0x00 len=3 st=0 err=0x00000000 data=12 34 FF
#00007 WRITE ptr=0x10 len=1 st=1 err=0x00000000 data=33
#00008 WRRD ptr=0x20 len=3 st=0 err=0x00000000 data=A2 1E FF
#00009 GCALL ptr=0x23 len=1 st=0 err=0x00000000 data=60
#00010 WRRD ptr=0x00 len=3 st=0 err=0x00000000 data=12 34 FF
--- log dump end ---
---- Sent utf8 encoded message: "stream_status\r\n" ----
--- stream status ---
id=0 addr=0x21 fill=1581/2048 free=467 underruns=0 drops=0 last=0x1E
---- Sent utf8 encoded message: "clear\r\n" ----
log cleared
---- Sent utf8 encoded message: "dump\r\n" ----
log is empty
```

10.4 Appendix D: STM32G474 Master Test Logs

This appendix contains the raw logs recorded by the STM32G474 I2C master test runner.

10.4.1 D.0 Test setup diagram

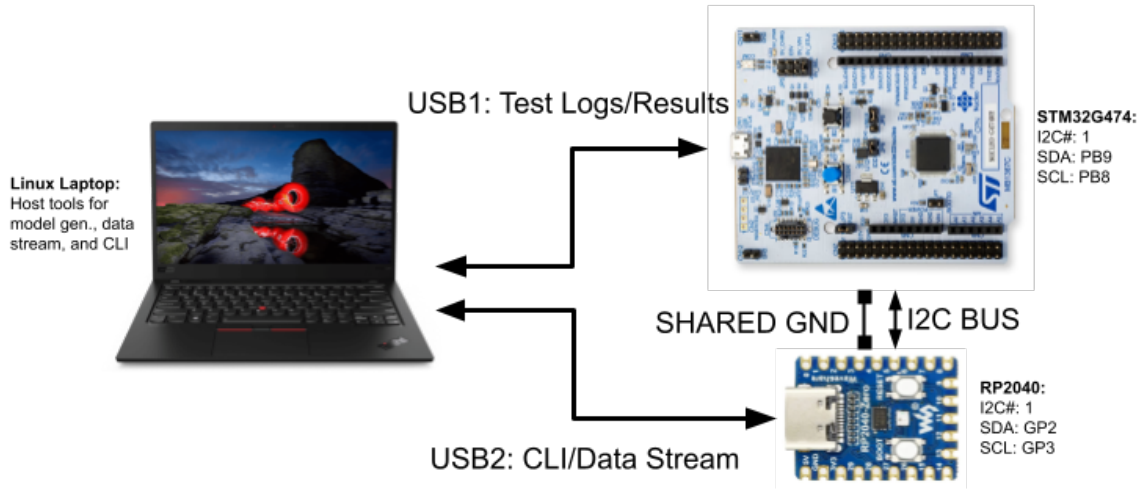


Figure 5: STM32G474-driven MorphIIC validation test setup.

10.4.2 D.1 Functional profile logs

Listing 7: Functional log: ai_off, stream_off.

```
Finished 'dev' profile [unoptimized + debuginfo] target(s) in 0.11s
Running 'probe-rs run --chip STM32G474RETx target/thumbv7em-none-eabihf/debug/morphiic-
functional-ai-off-stream-off'
Finished in 4.36s
0.000946 [INFO ] TEST_ACTION suite=functional_ai_off_stream_off id=START action=RUN detail=
functional_suite_begin model=auto_increment_false_model stream_mode=inactive auto_increment=
false (morphiic_functional_suite functional_suite/src/fmt.rs:133)
0.009033 [INFO ] TEST_ACTION suite=functional_ai_off_stream_off id=RATE action=SET rate_khz=100
status=OK (morphiic_functional_suite functional_suite/src/fmt.rs:133)
0.014373 [INFO ] TEST_CASE suite=functional_ai_off_stream_off id=F01_STATIC_DEFAULTS rate_khz=100
status=PASS total=1 passed=1 failed=0 skipped=0 note=static_defaults_ok (
morphiic_functional_suite functional_suite/src/fmt.rs:133)
0.027496 [INFO ] TEST_CASE suite=functional_ai_off_stream_off id=F02_RW_RO_MULTIBYTE rate_khz=100
status=PASS total=2 passed=2 failed=0 skipped=0 note=rw_ro_ok (morphiic_functional_suite
functional_suite/src/fmt.rs:133)
0.037780 [INFO ] TEST_CASE suite=functional_ai_off_stream_off id=F03_AUTO_INCREMENT_FLOW rate_khz
=100 status=PASS total=3 passed=3 failed=0 skipped=0 note=auto_increment_behavior_ok (
morphiic_functional_suite functional_suite/src/fmt.rs:133)
0.061828 [INFO ] TEST_CASE suite=functional_ai_off_stream_off id=F04_GENERAL_CALL_RESET rate_khz
=100 status=PASS total=4 passed=4 failed=0 skipped=0 note=reset_ok (morphiic_functional_suite
functional_suite/src/fmt.rs:133)
0.378601 [INFO ] TEST_CASE suite=functional_ai_off_stream_off id=F05_EMBEDDED_SEQ rate_khz=100
status=PASS total=5 passed=5 failed=0 skipped=0 note=embedded_cycle_ok (
morphiic_functional_suite functional_suite/src/fmt.rs:133)
```

```

0.447662 [INFO ] TEST_CASE suite=functional_ai_off_stream_off id=F06_HOST_EXPECT_INACTIVE rate_khz
=100 status=PASS total=6 passed=6 failed=0 skipped=0 note=host_fallback_constant (
morphiic_functional_suite functional_suite/src/fmt.rs:133)
0.458160 [INFO ] TEST_ACTION suite=functional_ai_off_stream_off id=RATE action=SET rate_khz=400
status=OK (morphiic_functional_suite functional_suite/src/fmt.rs:133)
0.463104 [INFO ] TEST_CASE suite=functional_ai_off_stream_off id=F01_STATIC_DEFAULTS rate_khz=400
status=PASS total=7 passed=7 failed=0 skipped=0 note=static_defaults_ok (
morphiic_functional_suite functional_suite/src/fmt.rs:133)
0.475036 [INFO ] TEST_CASE suite=functional_ai_off_stream_off id=F02_RW_RO_MULTIBYTE rate_khz=400
status=PASS total=8 passed=8 failed=0 skipped=0 note=rw_ro_ok (morphiic_functional_suite
functional_suite/src/fmt.rs:133)
0.484954 [INFO ] TEST_CASE suite=functional_ai_off_stream_off id=F03_AUTO_INCREMENT_FLOW rate_khz
=400 status=PASS total=9 passed=9 failed=0 skipped=0 note=auto_increment_behavior_ok (
morphiic_functional_suite functional_suite/src/fmt.rs:133)
0.508026 [INFO ] TEST_CASE suite=functional_ai_off_stream_off id=F04_GENERAL_CALL_RESET rate_khz
=400 status=PASS total=10 passed=10 failed=0 skipped=0 note=reset_ok (morphiic_functional_suite
functional_suite/src/fmt.rs:133)
0.721130 [INFO ] TEST_CASE suite=functional_ai_off_stream_off id=F05_EMBEDDED_SEQ rate_khz=400
status=PASS total=11 passed=11 failed=0 skipped=0 note=embedded_cycle_ok (
morphiic_functional_suite functional_suite/src/fmt.rs:133)
0.781433 [INFO ] TEST_CASE suite=functional_ai_off_stream_off id=F06_HOST_EXPECT_INACTIVE rate_khz
=400 status=PASS total=12 passed=12 failed=0 skipped=0 note=host_fallback_constant (
morphiic_functional_suite functional_suite/src/fmt.rs:133)
0.791931 [INFO ] TEST_SUMMARY suite=functional_ai_off_stream_off total=12 passed=12 failed=0
skipped=0 (morphiic_functional_suite functional_suite/src/fmt.rs:133)

```

Listing 8: Functional log: ai_off, stream_on.

```

Finished 'dev' profile [unoptimized + debuginfo] target(s) in 0.11s
Running 'probe-rs run --chip STM32G474RETx target/thumbv7em-none-eabi-hf/debug/morphiic-
functional-ai-off-stream-on'
Finished in 4.37s
0.000946 [INFO ] TEST_ACTION suite=functional_ai_off_stream_on id=START action=RUN detail=
functional_suite_begin model=auto_increment_false_model stream_mode=active auto_increment=false
(morphiic_functional_suite functional_suite/src/fmt.rs:133)
0.008697 [INFO ] TEST_ACTION suite=functional_ai_off_stream_on id=RATE action=SET rate_khz=100
status=OK (morphiic_functional_suite functional_suite/src/fmt.rs:133)
0.013977 [INFO ] TEST_CASE suite=functional_ai_off_stream_on id=F01_STATIC_DEFAULTS rate_khz=100
status=PASS total=1 passed=1 failed=0 skipped=0 note=static_defaults_ok (
morphiic_functional_suite functional_suite/src/fmt.rs:133)
0.027008 [INFO ] TEST_CASE suite=functional_ai_off_stream_on id=F02_RW_RO_MULTIBYTE rate_khz=100
status=PASS total=2 passed=2 failed=0 skipped=0 note=rw_ro_ok (morphiic_functional_suite
functional_suite/src/fmt.rs:133)
0.037200 [INFO ] TEST_CASE suite=functional_ai_off_stream_on id=F03_AUTO_INCREMENT_FLOW rate_khz
=100 status=PASS total=3 passed=3 failed=0 skipped=0 note=auto_increment_behavior_ok (
morphiic_functional_suite functional_suite/src/fmt.rs:133)
0.061157 [INFO ] TEST_CASE suite=functional_ai_off_stream_on id=F04_GENERAL_CALL_RESET rate_khz=100
status=PASS total=4 passed=4 failed=0 skipped=0 note=reset_ok (morphiic_functional_suite
functional_suite/src/fmt.rs:133)
0.377685 [INFO ] TEST_CASE suite=functional_ai_off_stream_on id=F05_EMBEDDED_SEQ rate_khz=100
status=PASS total=5 passed=5 failed=0 skipped=0 note=embedded_cycle_ok (
morphiic_functional_suite functional_suite/src/fmt.rs:133)
0.790039 [INFO ] TEST_CASE suite=functional_ai_off_stream_on id=F07_HOST_EXPECT_ACTIVE rate_khz=100
status=PASS total=6 passed=6 failed=0 skipped=0 note=feeder_activity_all_streams (
morphiic_functional_suite functional_suite/src/fmt.rs:133)
0.800689 [INFO ] TEST_ACTION suite=functional_ai_off_stream_on id=RATE action=SET rate_khz=400
status=OK (morphiic_functional_suite functional_suite/src/fmt.rs:133)
0.805541 [INFO ] TEST_CASE suite=functional_ai_off_stream_on id=F01_STATIC_DEFAULTS rate_khz=400

```

```

    status=PASS total=7 passed=7 failed=0 skipped=0 note=static_defaults_ok (
    morphiic_functional_suite functional_suite/src/fmt.rs:133)
0.817504 [INFO ] TEST_CASE suite=functional_ai_off_stream_on id=F02_RW_RO_MULTIBYTE rate_khz=400
    status=PASS total=8 passed=8 failed=0 skipped=0 note=rw_ro_ok (morphiic_functional_suite
    functional_suite/src/fmt.rs:133)
0.827301 [INFO ] TEST_CASE suite=functional_ai_off_stream_on id=F03_AUTO_INCREMENT_FLOW rate_khz
    =400 status=PASS total=9 passed=9 failed=0 skipped=0 note=auto_increment_behavior_ok (
    morphiic_functional_suite functional_suite/src/fmt.rs:133)
0.850372 [INFO ] TEST_CASE suite=functional_ai_off_stream_on id=F04_GENERAL_CALL_RESET rate_khz=400
    status=PASS total=10 passed=10 failed=0 skipped=0 note=reset_ok (morphiic_functional_suite
    functional_suite/src/fmt.rs:133)
1.063446 [INFO ] TEST_CASE suite=functional_ai_off_stream_on id=F05_EMBEDDED_SEQ rate_khz=400
    status=PASS total=11 passed=11 failed=0 skipped=0 note=embedded_cycle_ok (
    morphiic_functional_suite functional_suite/src/fmt.rs:133)
1.417633 [INFO ] TEST_CASE suite=functional_ai_off_stream_on id=F07_HOST_EXPECT_ACTIVE rate_khz=400
    status=PASS total=12 passed=12 failed=0 skipped=0 note=feeder_activity_all_streams (
    morphiic_functional_suite functional_suite/src/fmt.rs:133)
1.428283 [INFO ] TEST_SUMMARY suite=functional_ai_off_stream_on total=12 passed=12 failed=0 skipped
    =0 (morphiic_functional_suite functional_suite/src/fmt.rs:133)

```

Listing 9: Functional log: ai_on, stream_off.

```

Finished 'dev' profile [unoptimized + debuginfo] target(s) in 0.08s
Running 'probe-rs run --chip STM32G474RETx target/thumbv7em-none-eabi/hf/debug/morphiic-
functional-ai-on-stream-off'
Finished in 4.37s
0.000946 [INFO ] TEST_ACTION suite=functional_ai_on_stream_off id=START action=RUN detail=
    functional_suite_begin model=valid_max_everything stream_mode=inactive auto_increment=true (
    morphiic_functional_suite functional_suite/src/fmt.rs:133)
0.008361 [INFO ] TEST_ACTION suite=functional_ai_on_stream_off id=RATE action=SET rate_khz=100
    status=OK (morphiic_functional_suite functional_suite/src/fmt.rs:133)
0.013671 [INFO ] TEST_CASE suite=functional_ai_on_stream_off id=F01_STATIC_DEFAULTS rate_khz=100
    status=PASS total=1 passed=1 failed=0 skipped=0 note=static_defaults_ok (
    morphiic_functional_suite functional_suite/src/fmt.rs:133)
0.026702 [INFO ] TEST_CASE suite=functional_ai_on_stream_off id=F02_RW_RO_MULTIBYTE rate_khz=100
    status=PASS total=2 passed=2 failed=0 skipped=0 note=rw_ro_ok (morphiic_functional_suite
    functional_suite/src/fmt.rs:133)
0.036895 [INFO ] TEST_CASE suite=functional_ai_on_stream_off id=F03_AUTO_INCREMENT_FLOW rate_khz
    =100 status=PASS total=3 passed=3 failed=0 skipped=0 note=auto_increment_behavior_ok (
    morphiic_functional_suite functional_suite/src/fmt.rs:133)
0.060821 [INFO ] TEST_CASE suite=functional_ai_on_stream_off id=F04_GENERAL_CALL_RESET rate_khz=100
    status=PASS total=4 passed=4 failed=0 skipped=0 note=reset_ok (morphiic_functional_suite
    functional_suite/src/fmt.rs:133)
0.376098 [INFO ] TEST_CASE suite=functional_ai_on_stream_off id=F05_EMBEDDED_SEQ rate_khz=100
    status=PASS total=5 passed=5 failed=0 skipped=0 note=embedded_cycle_ok (
    morphiic_functional_suite functional_suite/src/fmt.rs:133)
0.444854 [INFO ] TEST_CASE suite=functional_ai_on_stream_off id=F06_HOST_EXPECT_INACTIVE rate_khz
    =100 status=PASS total=6 passed=6 failed=0 skipped=0 note=host_fallback_constant (
    morphiic_functional_suite functional_suite/src/fmt.rs:133)
0.455261 [INFO ] TEST_ACTION suite=functional_ai_on_stream_off id=RATE action=SET rate_khz=400
    status=OK (morphiic_functional_suite functional_suite/src/fmt.rs:133)
0.460113 [INFO ] TEST_CASE suite=functional_ai_on_stream_off id=F01_STATIC_DEFAULTS rate_khz=400
    status=PASS total=7 passed=7 failed=0 skipped=0 note=static_defaults_ok (
    morphiic_functional_suite functional_suite/src/fmt.rs:133)
0.472045 [INFO ] TEST_CASE suite=functional_ai_on_stream_off id=F02_RW_RO_MULTIBYTE rate_khz=400
    status=PASS total=8 passed=8 failed=0 skipped=0 note=rw_ro_ok (morphiic_functional_suite
    functional_suite/src/fmt.rs:133)
0.481872 [INFO ] TEST_CASE suite=functional_ai_on_stream_off id=F03_AUTO_INCREMENT_FLOW rate_khz

```

```

=400 status=PASS total=9 passed=9 failed=0 skipped=0 note=auto_increment_behavior_ok (
morphiic_functional_suite functional_suite/src/fmt.rs:133)
0.504852 [INFO ] TEST_CASE suite=functional_ai_on_stream_off id=F04_GENERAL_CALL_RESET rate_khz=400
status=PASS total=10 passed=10 failed=0 skipped=0 note=reset_ok (morphiic_functional_suite
functional_suite/src/fmt.rs:133)
0.717346 [INFO ] TEST_CASE suite=functional_ai_on_stream_off id=F05_EMBEDDED_SEQ rate_khz=400
status=PASS total=11 passed=11 failed=0 skipped=0 note=embedded_cycle_ok (
morphiic_functional_suite functional_suite/src/fmt.rs:133)
0.777648 [INFO ] TEST_CASE suite=functional_ai_on_stream_off id=F06_HOST_EXPECT_INACTIVE rate_khz
=400 status=PASS total=12 passed=12 failed=0 skipped=0 note=host_fallback_constant (
morphiic_functional_suite functional_suite/src/fmt.rs:133)
0.788055 [INFO ] TEST_SUMMARY suite=functional_ai_on_stream_off total=12 passed=12 failed=0 skipped
=0 (morphiic_functional_suite functional_suite/src/fmt.rs:133)

```

Listing 10: Functional log: ai_on, stream_on.

```

Finished 'dev' profile [unoptimized + debuginfo] target(s) in 0.08s
Running 'probe-rs run --chip STM32G474RETx target/thumbv7em-none-eabi-hf/debug/morphiic-
functional-ai-on-stream-on'
Finished in 4.54s
0.000946 [INFO ] TEST_ACTION suite=functional_ai_on_stream_on id=START action=RUN detail=
functional_suite_begin model=valid_max_everything stream_mode=active auto_increment=true (
morphiic_functional_suite functional_suite/src/fmt.rs:133)
0.008026 [INFO ] TEST_ACTION suite=functional_ai_on_stream_on id=RATE action=SET rate_khz=100
status=OK (morphiic_functional_suite functional_suite/src/fmt.rs:133)
0.013214 [INFO ] TEST_CASE suite=functional_ai_on_stream_on id=F01_STATIC_DEFAULTS rate_khz=100
status=PASS total=1 passed=1 failed=0 skipped=0 note=static_defaults_ok (
morphiic_functional_suite functional_suite/src/fmt.rs:133)
0.026153 [INFO ] TEST_CASE suite=functional_ai_on_stream_on id=F02_RW_RO_MULTIBYTE rate_khz=100
status=PASS total=2 passed=2 failed=0 skipped=0 note=rw_ro_ok (morphiic_functional_suite
functional_suite/src/fmt.rs:133)
0.036285 [INFO ] TEST_CASE suite=functional_ai_on_stream_on id=F03_AUTO_INCREMENT_FLOW rate_khz=100
status=PASS total=3 passed=3 failed=0 skipped=0 note=auto_increment_behavior_ok (
morphiic_functional_suite functional_suite/src/fmt.rs:133)
0.060150 [INFO ] TEST_CASE suite=functional_ai_on_stream_on id=F04_GENERAL_CALL_RESET rate_khz=100
status=PASS total=4 passed=4 failed=0 skipped=0 note=reset_ok (morphiic_functional_suite
functional_suite/src/fmt.rs:133)
0.376647 [INFO ] TEST_CASE suite=functional_ai_on_stream_on id=F05_EMBEDDED_SEQ rate_khz=100 status
=PASS total=5 passed=5 failed=0 skipped=0 note=embedded_cycle_ok (morphiic_functional_suite
functional_suite/src/fmt.rs:133)
0.789093 [INFO ] TEST_CASE suite=functional_ai_on_stream_on id=F07_HOST_EXPECT_ACTIVE rate_khz=100
status=PASS total=6 passed=6 failed=0 skipped=0 note=feeder_activity_all_streams (
morphiic_functional_suite functional_suite/src/fmt.rs:133)
0.799652 [INFO ] TEST_ACTION suite=functional_ai_on_stream_on id=RATE action=SET rate_khz=400
status=OK (morphiic_functional_suite functional_suite/src/fmt.rs:133)
0.804443 [INFO ] TEST_CASE suite=functional_ai_on_stream_on id=F01_STATIC_DEFAULTS rate_khz=400
status=PASS total=7 passed=7 failed=0 skipped=0 note=static_defaults_ok (
morphiic_functional_suite functional_suite/src/fmt.rs:133)
0.816284 [INFO ] TEST_CASE suite=functional_ai_on_stream_on id=F02_RW_RO_MULTIBYTE rate_khz=400
status=PASS total=8 passed=8 failed=0 skipped=0 note=rw_ro_ok (morphiic_functional_suite
functional_suite/src/fmt.rs:133)
0.826049 [INFO ] TEST_CASE suite=functional_ai_on_stream_on id=F03_AUTO_INCREMENT_FLOW rate_khz=400
status=PASS total=9 passed=9 failed=0 skipped=0 note=auto_increment_behavior_ok (
morphiic_functional_suite functional_suite/src/fmt.rs:133)
0.848968 [INFO ] TEST_CASE suite=functional_ai_on_stream_on id=F04_GENERAL_CALL_RESET rate_khz=400
status=PASS total=10 passed=10 failed=0 skipped=0 note=reset_ok (morphiic_functional_suite
functional_suite/src/fmt.rs:133)
1.062133 [INFO ] TEST_CASE suite=functional_ai_on_stream_on id=F05_EMBEDDED_SEQ rate_khz=400 status

```

```

=PASS total=11 passed=11 failed=0 skipped=0 note=embedded_cycle_ok (morphiic_functional_suite
functional_suite/src/fmt.rs:133)
1.416320 [INFO ] TEST_CASE suite=functional_ai_on_stream_on id=F07_HOST_EXPECT_ACTIVE rate_khz=400
status=PASS total=12 passed=12 failed=0 skipped=0 note=feeder_activity_all_streams (
morphiic_functional_suite functional_suite/src/fmt.rs:133)
1.426879 [INFO ] TEST_SUMMARY suite=functional_ai_on_stream_on total=12 passed=12 failed=0 skipped
=0 (morphiic_functional_suite functional_suite/src/fmt.rs:133)

```

10.4.3 D.2 Stress profile logs

Listing 11: Stress log: stress_ai_off.

```

Finished 'dev' profile [unoptimized + debuginfo] target(s) in 0.11s
Running 'probe-rs run --chip STM32G474RETx target/thumbv7em-none-eabi/hf/debug/morphiic-stress-
ai-off'
Finished in 4.63s
0.000946 [INFO ] TEST_ACTION suite=stress_ai_off id=START action=RUN detail=stress_suite_begin
model=valid_max_no_ai auto_increment=false (morphiic_stress_suite stress_suite/src/fmt.rs:133)
0.005645 [INFO ] TEST_ACTION suite=stress_ai_off id=S03 action=PREPARE detail=
start_csv_streamer_for_host_stream_tests model=/data/morphiic/models/max/valid_max_no_ai.json (
morphiic_stress_suite stress_suite/src/fmt.rs:133)
0.012115 [INFO ] TEST_ACTION suite=stress_ai_off id=S03 action=OPERATOR_NOTE detail=
ensure_csv_streamer_is_ON_before_S03_and_keep_it_ON_until_S04_stop_prompt (
morphiic_stress_suite stress_suite/src/fmt.rs:133)
0.014312 [INFO ] TEST_ACTION suite=stress_ai_off id=S05 action=CONFIG detail=ai_off_reads=2048
ai_on_bursts=2048 delay_ms=0 criterion=equal_adjacent_pairs>0 (morphiic_stress_suite
stress_suite/src/fmt.rs:133)
0.017517 [INFO ] TEST_ACTION suite=stress_ai_off id=RATE action=SET rate_khz=100 status=OK (
morphiic_stress_suite stress_suite/src/fmt.rs:133)
4.703704 [INFO ] S01 stats i2c_errors=0 invariant_failures=0 (morphiic_stress_suite stress_suite/
src/fmt.rs:133)
4.704986 [INFO ] TEST_CASE suite=stress_ai_off id=S01_MIXED_LOOP rate_khz=100 status=PASS total=1
passed=1 failed=0 skipped=0 note=mixed_loop_ok (morphiic_stress_suite stress_suite/src/fmt.rs
:133)
19.872741 [INFO ] S02 stats i2c_errors=0 invariant_failures=0 (morphiic_stress_suite stress_suite/
src/fmt.rs:133)
19.874084 [INFO ] TEST_CASE suite=stress_ai_off id=S02_BURST_LOOP rate_khz=100 status=PASS total=2
passed=2 failed=0 skipped=0 note=burst_loop_ok (morphiic_stress_suite stress_suite/src/fmt.rs
:133)
21.837371 [INFO ] S03 soak reg=0x10 unique=4 transitions=1023 has_seed=true (morphiic_stress_suite
stress_suite/src/fmt.rs:133)
21.839355 [INFO ] TEST_CASE suite=stress_ai_off id=S03_HOST_SOAK rate_khz=100 status=PASS total=3
passed=3 failed=0 skipped=0 note=host_soak_ok (morphiic_stress_suite stress_suite/src/fmt.rs
:133)
21.906219 [INFO ] TEST_ACTION suite=stress_ai_off id=S04 action=OPERATOR_BANNER detail
===== MANUAL_OPERATOR_STEP ===== (morphiic_stress_suite stress_suite/src/
fmt.rs:133)
21.909271 [INFO ] TEST_ACTION suite=stress_ai_off id=S04 action=OPERATOR_STEP detail=
turn_csv_streamer_OFF_now_then_wait_5s_then_wait_up_to_12s_for_plateau (morphiic_stress_suite
stress_suite/src/fmt.rs:133)
21.918518 [INFO ] TEST_ACTION suite=stress_ai_off id=S04 action=WAIT_AFTER_STOP detail=sleep_ms
=5000 (morphiic_stress_suite stress_suite/src/fmt.rs:133)
26.921417 [INFO ] TEST_ACTION suite=stress_ai_off id=S04 action=DRAIN_BEFORE_PLATEAU detail=reads
=2049 reg=0x10 (morphiic_stress_suite stress_suite/src/fmt.rs:133)
28.904327 [INFO ] TEST_ACTION suite=stress_ai_off id=S04 action=OPERATOR_BANNER detail
===== MANUAL_OPERATOR_STEP ===== (morphiic_stress_suite stress_suite/src/
fmt.rs:133)

```

```

28.907379 [INFO ] TEST_ACTION suite=stress_ai_off id=S04 action=OPERATOR_STEP detail=
    turn_csv_streamer_ON_now_then_wait_up_to_12s_for_resume (morphiic_stress_suite stress_suite/src
    /fmt.rs:133)
29.818481 [INFO ] TEST_CASE suite=stress_ai_off id=S04_STOP_RESUME rate_khz=100 status=PASS total=4
    passed=4 failed=0 skipped=0 note=stop_resume_ok (morphiic_stress_suite stress_suite/src/fmt.rs
    :133)
31.554382 [INFO ] TEST_ACTION suite=stress_ai_off id=S05B action=TIMING rate_khz=100 intervals=2047
    avg_inter_read_us=843 min_inter_read_us=762 max_inter_read_us=946 (morphiic_stress_suite
    stress_suite/src/fmt.rs:133)
31.558837 [INFO ] S05B single-reg reg=0x10 reads=2048 delay_ms=0 unique=4 transitions=2047
    equal_adjacent_pairs=0 repeat_streaks=0 longest_run=1 longest_run_value=0x0A saw_seed=true
    underrun_inferred=false avg_inter_read_us=843 min_inter_read_us=762 max_inter_read_us=946 (
    morphiic_stress_suite stress_suite/src/fmt.rs:133)
31.564086 [INFO ] TEST_CASE suite=stress_ai_off id=S05_SINGLE_REG_SEQ_UNDERRUN_AI_OFF rate_khz=100
    status=PASS total=5 passed=5 failed=0 skipped=0 note=underrun_not_observed (
    morphiic_stress_suite stress_suite/src/fmt.rs:133)
31.574157 [INFO ] TEST_ACTION suite=stress_ai_off id=RATE action=SET rate_khz=400 status=OK (
    morphiic_stress_suite stress_suite/src/fmt.rs:133)
34.379486 [INFO ] S01 stats i2c_errors=0 invariant_failures=0 (morphiic_stress_suite stress_suite/
    src/fmt.rs:133)
34.380798 [INFO ] TEST_CASE suite=stress_ai_off id=S01_MIXED_LOOP rate_khz=400 status=PASS total=6
    passed=6 failed=0 skipped=0 note=mixed_loop_ok (morphiic_stress_suite stress_suite/src/fmt.rs
    :133)
42.432342 [INFO ] S02 stats i2c_errors=0 invariant_failures=0 (morphiic_stress_suite stress_suite/
    src/fmt.rs:133)
42.433685 [INFO ] TEST_CASE suite=stress_ai_off id=S02_BURST_LOOP rate_khz=400 status=PASS total=7
    passed=7 failed=0 skipped=0 note=burst_loop_ok (morphiic_stress_suite stress_suite/src/fmt.rs
    :133)
44.112487 [INFO ] S03 soak reg=0x10 unique=4 transitions=1023 has_seed=true (morphiic_stress_suite
    stress_suite/src/fmt.rs:133)
44.114471 [INFO ] TEST_CASE suite=stress_ai_off id=S03_HOST_SOAK rate_khz=400 status=PASS total=8
    passed=8 failed=0 skipped=0 note=host_soak_ok (morphiic_stress_suite stress_suite/src/fmt.rs
    :133)
44.122039 [INFO ] TEST_CASE suite=stress_ai_off id=S04_STOP_RESUME rate_khz=400 status=SKIP total=9
    passed=8 failed=0 skipped=1 note>manual_case_only_at_100khz (morphiic_stress_suite
    stress_suite/src/fmt.rs:133)
45.311035 [INFO ] TEST_ACTION suite=stress_ai_off id=S05B action=TIMING rate_khz=400 intervals=2047
    avg_inter_read_us=575 min_inter_read_us=549 max_inter_read_us=701 (morphiic_stress_suite
    stress_suite/src/fmt.rs:133)
45.315551 [INFO ] S05B single-reg reg=0x10 reads=2048 delay_ms=0 unique=4 transitions=2047
    equal_adjacent_pairs=0 repeat_streaks=0 longest_run=1 longest_run_value=0x0A saw_seed=true
    underrun_inferred=false avg_inter_read_us=575 min_inter_read_us=549 max_inter_read_us=701 (
    morphiic_stress_suite stress_suite/src/fmt.rs:133)
45.320770 [INFO ] TEST_CASE suite=stress_ai_off id=S05_SINGLE_REG_SEQ_UNDERRUN_AI_OFF rate_khz=400
    status=PASS total=10 passed=9 failed=0 skipped=1 note=underrun_not_observed (
    morphiic_stress_suite stress_suite/src/fmt.rs:133)
45.330932 [INFO ] TEST_SUMMARY suite=stress_ai_off total=10 passed=9 failed=0 skipped=1 (
    morphiic_stress_suite stress_suite/src/fmt.rs:133)

```

Listing 12: Stress log: stress_ai_on.

```

0.000946 [INFO ] TEST_ACTION suite=stress_ai_on id=START action=RUN detail=stress_suite_begin model
    =valid_max_ai auto_increment=true (morphiic_stress_suite stress_suite/src/fmt.rs:133)
0.005218 [INFO ] TEST_ACTION suite=stress_ai_on id=S03 action=PREPARE detail=
    start_csv_streamer_for_host_stream_tests model=/data/morphiic/models/max/valid_max_ai.json (
    morphiic_stress_suite stress_suite/src/fmt.rs:133)
0.011352 [INFO ] TEST_ACTION suite=stress_ai_on id=S03 action=OPERATOR_NOTE detail=
    ensure_csv_streamer_is_ON_before_S03_and_keep_it_ON_until_S04_stop_prompt (

```

```

    morphiic_stress_suite stress_suite/src/fmt.rs:133)
0.013458 [INFO ] TEST_ACTION suite=stress_ai_on id=S05 action=CONFIG detail=ai_off_reads=2048
    ai_on_bursts=2048 delay_ms=0 criterion=equal_adjacent_pairs>0 (morphiic_stress_suite
    stress_suite/src/fmt.rs:133)
0.016571 [INFO ] TEST_ACTION suite=stress_ai_on id=RATE action=SET rate_khz=100 status=OK (
    morphiic_stress_suite stress_suite/src/fmt.rs:133)
14.966461 [INFO ] S01 stats i2c_errors=0 invariant_failures=0 (morphiic_stress_suite stress_suite/
    src/fmt.rs:133)
14.967712 [INFO ] TEST_CASE suite=stress_ai_on id=S01_MIXED_LOOP rate_khz=100 status=PASS total=1
    passed=1 failed=0 skipped=0 note=mixed_loop_ok (morphiic_stress_suite stress_suite/src/fmt.rs
    :133)
79.390716 [INFO ] S02 stats i2c_errors=0 invariant_failures=0 (morphiic_stress_suite stress_suite/
    src/fmt.rs:133)
79.392059 [INFO ] TEST_CASE suite=stress_ai_on id=S02_BURST_LOOP rate_khz=100 status=PASS total=2
    passed=2 failed=0 skipped=0 note=burst_loop_ok (morphiic_stress_suite stress_suite/src/fmt.rs
    :133)
81.358276 [INFO ] S03 soak reg=0x10 unique=4 transitions=1023 has_seed=true (morphiic_stress_suite
    stress_suite/src/fmt.rs:133)
81.360260 [INFO ] TEST_CASE suite=stress_ai_on id=S03_HOST_SOAK rate_khz=100 status=PASS total=3
    passed=3 failed=0 skipped=0 note=host_soak_ok (morphiic_stress_suite stress_suite/src/fmt.rs
    :133)
81.427032 [INFO ] TEST_ACTION suite=stress_ai_on id=S04 action=OPERATOR_BANNER detail
    ===== MANUAL_OPERATOR_STEP ===== (morphiic_stress_suite stress_suite/src/
    fmt.rs:133)
81.430023 [INFO ] TEST_ACTION suite=stress_ai_on id=S04 action=OPERATOR_STEP detail=
    turn_csv_streamer_OFF_now_then_wait_5s_then_wait_up_to_12s_for_plateau (morphiic_stress_suite
    stress_suite/src/fmt.rs:133)
81.439178 [INFO ] TEST_ACTION suite=stress_ai_on id=S04 action=WAIT_AFTER_STOP detail=sleep_ms=5000
    (morphiic_stress_suite stress_suite/src/fmt.rs:133)
86.441955 [INFO ] TEST_ACTION suite=stress_ai_on id=S04 action=DRAIN_BEFORE_PLATEAU detail=reads
    =2049 reg=0x10 (morphiic_stress_suite stress_suite/src/fmt.rs:133)
88.434906 [INFO ] TEST_ACTION suite=stress_ai_on id=S04 action=OPERATOR_BANNER detail
    ===== MANUAL_OPERATOR_STEP ===== (morphiic_stress_suite stress_suite/src/
    fmt.rs:133)
88.437896 [INFO ] TEST_ACTION suite=stress_ai_on id=S04 action=OPERATOR_STEP detail=
    turn_csv_streamer_ON_now_then_wait_up_to_12s_for_resume (morphiic_stress_suite stress_suite/src
    /fmt.rs:133)
90.110687 [INFO ] TEST_CASE suite=stress_ai_on id=S04_STOP_RESUME rate_khz=100 status=PASS total=4
    passed=4 failed=0 skipped=0 note=stop_resume_ok (morphiic_stress_suite stress_suite/src/fmt.rs
    :133)
94.286041 [INFO ] S05A stream=0x10 bursts=2048 delay_ms=0 unique=4 transitions=2047
    equal_adjacent_pairs=0 repeat_streaks=0 longest_run=1 longest_run_value=0x0A saw_seed=true (
    morphiic_stress_suite stress_suite/src/fmt.rs:133)
94.289764 [INFO ] S05A stream=0x11 bursts=2048 delay_ms=0 unique=4 transitions=2047
    equal_adjacent_pairs=0 repeat_streaks=0 longest_run=1 longest_run_value=0x0A saw_seed=true (
    morphiic_stress_suite stress_suite/src/fmt.rs:133)
94.293487 [INFO ] S05A stream=0x12 bursts=2048 delay_ms=0 unique=4 transitions=2047
    equal_adjacent_pairs=0 repeat_streaks=0 longest_run=1 longest_run_value=0x0A saw_seed=true (
    morphiic_stress_suite stress_suite/src/fmt.rs:133)
94.297210 [INFO ] S05A stream=0x13 bursts=2048 delay_ms=0 unique=4 transitions=2047
    equal_adjacent_pairs=0 repeat_streaks=0 longest_run=1 longest_run_value=0x0A saw_seed=true (
    morphiic_stress_suite stress_suite/src/fmt.rs:133)
94.300933 [INFO ] S05A stream=0x14 bursts=2048 delay_ms=0 unique=4 transitions=2047
    equal_adjacent_pairs=0 repeat_streaks=0 longest_run=1 longest_run_value=0x0A saw_seed=true (
    morphiic_stress_suite stress_suite/src/fmt.rs:133)
94.304626 [INFO ] S05A stream=0x15 bursts=2048 delay_ms=0 unique=4 transitions=2047
    equal_adjacent_pairs=0 repeat_streaks=0 longest_run=1 longest_run_value=0x0A saw_seed=true (
    morphiic_stress_suite stress_suite/src/fmt.rs:133)
94.308349 [INFO ] S05A stream=0x16 bursts=2048 delay_ms=0 unique=4 transitions=2047

```

```

    equal_adjacent_pairs=0 repeat_streaks=0 longest_run=1 longest_run_value=0x0A saw_seed=true (
    morphiic_stress_suite stress_suite/src/fmt.rs:133)
94.312072 [INFO ] S05A stream=0x17 bursts=2048 delay_ms=0 unique=4 transitions=2047
    equal_adjacent_pairs=0 repeat_streaks=0 longest_run=1 longest_run_value=0x0A saw_seed=true (
    morphiic_stress_suite stress_suite/src/fmt.rs:133)
94.315795 [INFO ] S05A stream=0x18 bursts=2048 delay_ms=0 unique=4 transitions=2047
    equal_adjacent_pairs=0 repeat_streaks=0 longest_run=1 longest_run_value=0x0A saw_seed=true (
    morphiic_stress_suite stress_suite/src/fmt.rs:133)
94.319519 [INFO ] TEST_ACTION suite=stress_ai_on id=S05A action=TIMING rate_khz=100 intervals=2047
    avg_inter_read_us=2034 min_inter_read_us=1953 max_inter_read_us=2166 (morphiic_stress_suite
    stress_suite/src/fmt.rs:133)
94.323852 [INFO ] S05A summary streams_with_seed=9 streams_with_repeat=0 total_equal_adjacent_pairs
    =0 max_longest_run=1 underrun_inferred=false avg_inter_read_us=2034 min_inter_read_us=1953
    max_inter_read_us=2166 (morphiic_stress_suite stress_suite/src/fmt.rs:133)
94.327362 [INFO ] TEST_CASE suite=stress_ai_on id=S05_RING_SEQ_UNDERRUN_AI_ON rate_khz=100 status=
    PASS total=5 passed=5 failed=0 skipped=0 note=underrun_not_observed (morphiic_stress_suite
    stress_suite/src/fmt.rs:133)
94.336730 [INFO ] TEST_ACTION suite=stress_ai_on id=RATE action=SET rate_khz=400 status=OK (
    morphiic_stress_suite stress_suite/src/fmt.rs:133)
104.265014 [INFO ] S01 stats i2c_errors=0 invariant_failures=0 (morphiic_stress_suite stress_suite/
    src/fmt.rs:133)
104.266357 [INFO ] TEST_CASE suite=stress_ai_on id=S01_MIXED_LOOP rate_khz=400 status=PASS total=6
    passed=6 failed=0 skipped=0 note=mixed_loop_ok (morphiic_stress_suite stress_suite/src/fmt.rs
    :133)
146.502929 [INFO ] S02 stats i2c_errors=0 invariant_failures=0 (morphiic_stress_suite stress_suite/
    src/fmt.rs:133)
146.504272 [INFO ] TEST_CASE suite=stress_ai_on id=S02_BURST_LOOP rate_khz=400 status=PASS total=7
    passed=7 failed=0 skipped=0 note=burst_loop_ok (morphiic_stress_suite stress_suite/src/fmt.rs
    :133)
148.185180 [INFO ] S03 soak reg=0x10 unique=4 transitions=1023 has_seed=true (morphiic_stress_suite
    stress_suite/src/fmt.rs:133)
148.187194 [INFO ] TEST_CASE suite=stress_ai_on id=S03_HOST_SOAK rate_khz=400 status=PASS total=8
    passed=8 failed=0 skipped=0 note=host_soak_ok (morphiic_stress_suite stress_suite/src/fmt.rs
    :133)
148.194641 [INFO ] TEST_CASE suite=stress_ai_on id=S04_STOP_RESUME rate_khz=400 status=SKIP total=9
    passed=8 failed=0 skipped=1 note>manual_case_only_at_100khz (morphiic_stress_suite
    stress_suite/src/fmt.rs:133)
150.813385 [INFO ] S05A stream=0x10 bursts=2048 delay_ms=0 unique=4 transitions=2047
    equal_adjacent_pairs=0 repeat_streaks=0 longest_run=1 longest_run_value=0x0A saw_seed=true (
    morphiic_stress_suite stress_suite/src/fmt.rs:133)
150.817108 [INFO ] S05A stream=0x11 bursts=2048 delay_ms=0 unique=4 transitions=2047
    equal_adjacent_pairs=0 repeat_streaks=0 longest_run=1 longest_run_value=0x0A saw_seed=true (
    morphiic_stress_suite stress_suite/src/fmt.rs:133)
150.820831 [INFO ] S05A stream=0x12 bursts=2048 delay_ms=0 unique=4 transitions=2047
    equal_adjacent_pairs=0 repeat_streaks=0 longest_run=1 longest_run_value=0x0A saw_seed=true (
    morphiic_stress_suite stress_suite/src/fmt.rs:133)
150.824523 [INFO ] S05A stream=0x13 bursts=2048 delay_ms=0 unique=4 transitions=2047
    equal_adjacent_pairs=0 repeat_streaks=0 longest_run=1 longest_run_value=0x0A saw_seed=true (
    morphiic_stress_suite stress_suite/src/fmt.rs:133)
150.828247 [INFO ] S05A stream=0x14 bursts=2048 delay_ms=0 unique=4 transitions=2047
    equal_adjacent_pairs=0 repeat_streaks=0 longest_run=1 longest_run_value=0x0A saw_seed=true (
    morphiic_stress_suite stress_suite/src/fmt.rs:133)
150.831970 [INFO ] S05A stream=0x15 bursts=2048 delay_ms=0 unique=4 transitions=2047
    equal_adjacent_pairs=0 repeat_streaks=0 longest_run=1 longest_run_value=0x0A saw_seed=true (
    morphiic_stress_suite stress_suite/src/fmt.rs:133)
150.835693 [INFO ] S05A stream=0x16 bursts=2048 delay_ms=0 unique=4 transitions=2047
    equal_adjacent_pairs=0 repeat_streaks=0 longest_run=1 longest_run_value=0x0A saw_seed=true (
    morphiic_stress_suite stress_suite/src/fmt.rs:133)
150.839416 [INFO ] S05A stream=0x17 bursts=2048 delay_ms=0 unique=4 transitions=2047

```

```

    equal_adjacent_pairs=0 repeat_streaks=0 longest_run=1 longest_run_value=0x0A saw_seed=true (
    morphiic_stress_suite stress_suite/src/fmt.rs:133)
150.843109 [INFO ] S05A stream=0x18 bursts=2048 delay_ms=0 unique=4 transitions=2047
    equal_adjacent_pairs=0 repeat_streaks=0 longest_run=1 longest_run_value=0x0A saw_seed=true (
    morphiic_stress_suite stress_suite/src/fmt.rs:133)
150.846832 [INFO ] TEST_ACTION suite=stress_ai_on id=S05A action=TIMING rate_khz=400 intervals=2047
    avg_inter_read_us=1273 min_inter_read_us=1220 max_inter_read_us=1434 (morphiic_stress_suite
    stress_suite/src/fmt.rs:133)
150.851257 [INFO ] S05A summary streams_with_seed=9 streams_with_repeat=0
    total_equal_adjacent_pairs=0 max_longest_run=1 underrun_inferred=false avg_inter_read_us=1273
    min_inter_read_us=1220 max_inter_read_us=1434 (morphiic_stress_suite stress_suite/src/fmt.rs
    :133)
150.854766 [INFO ] TEST_CASE suite=stress_ai_on id=S05_RING_SEQ_UNDERRUN_AI_ON rate_khz=400 status=
    PASS total=10 passed=9 failed=0 skipped=1 note=underrun_not_observed (morphiic_stress_suite
    stress_suite/src/fmt.rs:133)
150.864257 [INFO ] TEST_SUMMARY suite=stress_ai_on total=10 passed=9 failed=0 skipped=1 (
    morphiic_stress_suite stress_suite/src/fmt.rs:133)

```